

How to Tolerate Typos in Strong Asymmetric PAKE

Ian McQuoid*

Mike Rosulek*

Jiayu Xu*

July 30, 2025

Abstract

Strong asymmetric password-authenticated key exchange (saPAKE) is the gold standard for password-based authentication. When authenticating using saPAKE, the client holds a cleartext password, and the server holds only a “digest” of the password. The two parties obtain a shared session key if and only if the client password matches the password encoded in the digest.

In this work we initiate the study of **strong asymmetric fuzzy PAKE (safPAKE)**, which allows the client and server to obtain a shared session key if the client’s password is “close enough” to the password encoded in the digest, according to some policy. safPAKE can be used to tolerate incidental password typos in the PAKE setting, which is becoming a standard industry practice outside the PAKE setting. Our safPAKE functionality supports any “typo policy”, and our protocol is practical when there are a small number of permissible mistypings of a password.

1 Introduction

Password-based authentication is delicate because passwords are low entropy. Cryptographic mechanisms that are safe when using high-entropy secrets can be unsuitable for passwords, allowing an attacker to *locally* check whether a password guess is correct after the fact. Such a mechanism will expose typical passwords after mounting an offline dictionary attack. Password-authenticated key exchange (PAKE) [BM93] was proposed as a cryptographic mechanism disallowing such offline attacks.

PAKE is an improvement over the password-over-TLS approach currently used on the Internet, in which a client sends (an encryption of) its password over a TLS channel, and the server, upon decryption, checks raw passwords against a password file stored in advance. The security of password-over-TLS critically relies on the assumption that the server’s public key is correctly distributed to the client; however, numerous instances of “PKI failure” in recent years¹ clearly show that a password protocol without PKI — the setting of PAKE — is preferable.

Password-authenticated key exchange (PAKE). PAKE is a family of protocols that allow two parties to agree on a common session key if and only if they have matching passwords. PAKE protocols are carefully designed to avoid exposing offline dictionary attacks as the shared strings (the passwords) are assumed to be low-entropy and therefore the space of secret strings is efficiently enumerable. Instead, they achieve the best possible security: the only way for an adversary to check a password guess is by participating in an interactive protocol session.

There are two main flavors of PAKE:

*Oregon State University {mcquoidi,rosulekm,xujiay}@oregonstate.edu. Second author partially supported by NSF award 2150726.

¹Amir Herzberg has previously provided a list of famous PKI failures [Her22].

- **Symmetric PAKE** [BM93, BPR00, BMP00, KOY01, KP01, AP05, CHK⁺05, KV11, MRR20] requires both parties to hold the passwords in cleartext.
- **Asymmetric PAKE** [BM93, GMR06, Sho20, GJK21, SGJK22] is more suitable for a client-server setting, where the server holds only a one-way digest of the password.

Asymmetric PAKE has the additional property that, even in the setting where the server’s password file might be compromised, an adversary must perform an (offline) dictionary attack to recover any passwords. In other words, the server cannot store its passwords in the clear, and furthermore never sees the raw passwords *throughout the execution of the protocol*, which is another advantage over the password-over-TLS approach. The gold standard security definition for asymmetric PAKE is **strong asymmetric PAKE (saPAKE)** [JKX18a, BJX19] where, in addition to the asymmetric PAKE security requirements, computation for the offline dictionary attack must be performed *after* compromising the server. This distinction is analogous to the server holding a digest $H(pw)$ for a public hash function H vs the server holding a digest $H(s; pw)$ for some uniform and secret “salt” s . In the first case (aPAKE), all possible digests can be computed before server compromise at which point the adversary immediately learns the correct password; while in the second case (saPAKE), the adversary must wait until server compromise before launching an offline attack.

The original security notion for PAKE is game-based [BPR00]. In recent years, the standard has switched to the stronger notion in the Universal Composability (UC) framework [CHK⁺05, GMR06, JKC18a], which supports arbitrary composition of the protocol (with itself or with other protocols). Variants of the original UC PAKE notion have been proposed, including relaxations of the UC functionality [ABB⁺20] and combining the security notion with idealized models [ABK⁺21].

Password typos. Human-typed passwords are prone to typos, which prevent successful authentication. Some online services (in which the server sees the login password in the clear) are known to correct for some password typos. For example, Facebook accepts passwords with inverted upper/lowercase (e.g., erroneous Caps Lock) or an erroneously capitalized first letter (possible in many input methods on mobile phones) [Pro11].

Chatterjee *et al.* [CAA⁺16] were the first to formally study the security implications of correcting minor typos — *i.e.*, allowing users to authenticate even if they make certain incidental typos. In their study of real-world login attempts to Dropbox, they found that 20% of users that experienced a failed login attempt would have saved at least 1 minute of time if simple typo-correction had been enabled. They found that 3% of users that experienced a failed login attempt were never able to successfully log in, even though they used a password that would have been accepted under a simple typo-correction scheme.

Tolerating typos in PAKE. It is relatively trivial to tolerate typos when the server sees the user’s password attempt in the clear — *i.e.*, when authenticating with password-over-TLS. It is a challenge to tolerate typos in the PAKE setting, when neither party sees the other’s password attempt. Previous constructions of fuzzy PAKE variants have achieved varying levels of security. Dupont *et al.* [DHP⁺18] constructed two fuzzy-input tolerant symmetric PAKE protocols, one from garbled circuits and oblivious transfer which can handle “closeness” for any efficiently computable circuit (in the language of our construction, any efficiently computable similarity predicate) and one from PAKE which can handle fuzzy inputs in a small hamming ball. The former is a feasibility result that is inefficient, whereas the latter was later found to be insecure [BFH⁺23]; [BFH⁺23] also presented another fuzzy symmetric PAKE protocol.

Moving the result for fuzzy PAKE to cover asymmetric security, Erwig *et al.* [EHOR20] constructed two fuzzy-input tolerant asymmetric PAKE protocols, one from error correcting codes

and oblivious transfer which can handle fuzzy inputs in a hamming ball and the second which compiled aPAKE into a fuzzy protocol which achieves a weaker functionality but can handle fixed size closeness notions. Namely, the functionality has been weakened to allow for a fixed number of unstructured guesses in an online attack. Pongmorrakot and Chatterjee [PC20] similarly construct a fuzzy PAKE protocol which has security guarantees in the face of server compromise. However, their result similarly allows the server to make a fixed number of unstructured guesses by design, and allows the adversary to impersonate the user on compromise of the server — something which is not standard for aPAKE protocols.

Crucially, none of the results above achieves *strong* asymmetric security; even the fuzzy aPAKE protocols allow the adversary to perform some computation before server compromise, and recover the password immediately once the server is compromised. This is the starting point of our work:

Can we construct a typo-tolerant strong aPAKE protocol resilient to pre-computation attacks, preferably with reasonable efficiency?

Naïve approach. Let Σ_{pw} denote the set of allowable typos of the correct password pw . The server would like to authenticate any user who enters a password pw' from Σ_{pw} . The standard PAKE setting corresponds to the special case $\Sigma_{pw} = \{pw\}$.

A natural way to tolerate typos in saPAKE is to run t independent instances of saPAKE, where $t = |\Sigma_{pw}|$. In more detail: the server holds a password digest for each $\tilde{pw} \in \Sigma_{pw}$. The client holds a password attempt pw' and tries to authenticate t times — once against each of these digests.

This naïve approach to typo tolerance has two main drawbacks:

- Consider what would be the best possible security for a single authentication attempt of typo-tolerant saPAKE: A corrupt client should learn whether $pw' \in \Sigma_{pw}$, for *only one* guess pw' . A corrupt server should learn whether $pw' \in \Sigma_{pw}$ for *only one* guess pw .

The naïve protocol leaks more than this. Namely, a corrupt client can learn whether $pw'_i \in \Sigma_{pw}$ for t different and *arbitrary* guesses pw'_i . A corrupt server can learn whether $pw' \in S$ for a *completely unstructured* set S of cardinality t (rather than a “typo ball” Σ_{pw} around a single center pw). In other words, the naïve protocol makes online dictionary attacks t times faster (in terms of number of authentication attempts), for either party.

Fixing the issue for a corrupt client is relatively straight-forward. But it is non-trivial to prevent a corrupt server from making t *unstructured* guesses per authentication attempt.

- All costs of the protocol scale with the number t of allowed typos. For most password-typo policies (described above), t is indeed a very small constant, so this overhead is not prohibitive. But it is nontrivial to make any aspect of the protocol’s complexity scale sublinearly with t .

1.1 Our Results

We show how to resolve both of the problems described above by giving an efficient protocol for **strong asymmetric fuzzy PAKE (safPAKE)**. Our protocol achieves best-possible security (with respect to password guesses) in the Universal Composability (UC) framework, with communication independent of t for a wide definition of “closeness” which includes a common subclass of the similarity predicates considered by Erwig et al. [EHOR20]. Our construction can be viewed as an extension of the saPAKE protocol by Jarecki, Krawczyk, and Xu [JKX18a], which is a compiler from any UC oblivious PRF (OPRF) and any UC aPAKE to UC saPAKE²; the main difference

²The provided security proof in [JKX18a] has not been updated to their new saPAKE security model.

is that we add a one-round Private Information Retrieval (PIR) protocol between the OPRF and aPAKE stages. The saPAKE protocol in [JKX18a] can be viewed as a special case of our protocol with the equality check function for a similarity predicate and the usual PAKE leakage functions.

Additionally, our protocol up to the aPAKE stage can be seen independently as an input-normalization protocol where a client inputs a string x' and returns a “corrected” version of their string x . This normalization achieves the flavor of security needed for saPAKE — namely, resistance to online dictionary attacks and to precomputation for offline dictionary attacks. We believe that this conceptual contribution may find independent applications elsewhere.

A final contribution is to modify the 2-hash-Diffie-Hellman (2HDH) OPRF protocol of [JKKX17, JKX18a] to support a kind of key-extraction property that is necessary for our safPAKE protocol. As our OPRF functionality is key-aware and our OPRF protocol allows for key extraction, we believe that our safPAKE protocol is amenable to an interactive registration phase requiring only a single message and without the honest server receiving the client’s clear-text password. As a comparison, an interactive registration for OPAQUE [JKX18a] would require at least three messages as the OPRF would need to be evaluated in an online step. This is because OPAQUE’s OPRF functionality does not have a notion of transferrable OPRF keys³. We leave modeling interactive registration in aPAKE, and its extensions, as future work.

2 Preliminaries

2.1 (Keyword) PIR

Our safPAKE protocol requires a (single-server) keyword PIR protocol. In keyword PIR, the server holds a key-value mapping $\text{DB} : u_i \mapsto v_i$ and the client holds a key u . The outcome of the protocol is that the client learns (at least) $v := \text{DB}[u]$, if it exists, while the server learns nothing about u .

We write our safPAKE protocol in terms of a 1-round PIR protocol for the sake of simplicity, although our protocol can be easily extended to use a multi-round PIR. Formally, a (1-round) keyword PIR protocol consists of the following algorithms:

- $\text{PIR.Request}(u) \rightarrow (s, \text{req})$
- $\text{PIR.Respond}(\text{req}, \text{DB}) \rightarrow \text{resp}$
- $\text{PIR.Finish}(s, \text{resp}) \rightarrow v$

It must satisfy the following properties:

- **Correctness:** For all key-value mappings DB , and for all keys u appearing in DB , the following returns true with overwhelming probability:

$$\begin{aligned} (s, \text{req}) &\leftarrow \text{PIR.Request}(u) \\ \text{resp} &:= \text{PIR.Respond}(\text{req}, \text{DB}) \\ \text{return } \text{DB}[u] &= \text{PIR.Finish}(\text{resp}, s) \end{aligned}$$

There is no correctness required when u is not a key appearing in DB .

- **Client Privacy:** For all u, u' , the following distributions are indistinguishable:

$$\begin{aligned} (s, \text{req}) &\leftarrow \text{PIR.Request}(u) \\ \text{return req} \end{aligned} \approx \begin{aligned} (s', \text{req}) &\leftarrow \text{PIR.Request}(u') \\ \text{return req} \end{aligned}$$

³However, we provide evidence that an OPAQUE analog leveraging our OPRF functionality is secure and, therefore, OPAQUE would likely enjoy a single-message interactive registration phase.

Our protocol requires a **deterministic** PIR.Respond algorithm, which is standard since PIR.Respond is involved only in correctness, but not privacy.

Candidate instantiations. In **trivial PIR**, req is empty and $\text{PIR.Respond} := \text{DB}$.

A classic keyword PIR protocol has $\text{req} := \text{Enc}(s, u)$, for a symmetric-key fully homomorphic encryption scheme. The server interpolates a polynomial P such that $P(u) = \text{DB}[u]$ for all $u \in \text{DB}$. It then homomorphically evaluates the polynomial on req to obtain an encryption of $P(u) = \text{DB}[u]$.

2.2 Authenticated Encryption

Let $\mathcal{E} = (\text{Enc}, \text{Dec})$ be a symmetric encryption scheme, with key space $\mathcal{K}$, and where plaintexts of length n are encrypted to ciphertexts in the set $\mathcal{C}(n)$.

Definition 1. We say that $\mathcal{E}$ satisfies **authenticated encryption (AE\$)** security if the following two stateful oracles are computationally indistinguishable:

$ \begin{array}{l} k \leftarrow \mathcal{K} \\ \text{enc}(m \in \mathcal{M}): \\ \quad \text{return } \text{Enc}(k, m) \\ \text{dec}(c \in \mathcal{C}): \\ \quad \text{return } \text{Dec}(k, c) \end{array} $	$\approx$	<pre> // $\perp$ is the default value for undefined keys ptxts[] := $\perp$ enc($m \in \mathcal{M}$): $c \leftarrow \mathcal{C}(m)$ ptxts[c] := m return c dec($c \in \mathcal{C}$): return ptxts[c] </pre>
---	-----------	--

3 Oblivious PRF

Our protocol requires an Oblivious PRF (OPRF), but prior OPRF ideal functionalities [JKX18a, JKK14, JKKX17] from related work are not suitable for our purposes. In our protocol, when an honest client authenticates using a valid password, she learns the server’s OPRF key. This key gives her the ability to evaluate the OPRF without the server’s help, which is necessary for some consistency checks in the protocol.

Existing OPRF functionalities do not explicitly have the notion of a “key” as a string that can be transferred or encrypted. In the OPAQUE protocol for instance, only an adversary can learn the server’s OPRF key — when it compromises the server storage. This is modeled in the protocol by the adversary sending an explicit “compromise” command to the OPRF functionality, which grants the adversary the ability to evaluate the OPRF.

In this section, we describe an ideal OPRF functionality and corresponding protocol that supports transferrable PRF keys. The protocol is based on the 2-hash-Diffie-Hellman (2HDH) OPRF protocol of [JKKX17]. The main challenge in our setting is identifying which key a corrupt server is using in a specific OPRF interaction. This is a challenge because the key is a Diffie-Hellman exponent and thus difficult for the simulator to extract. We accomplish this by strategically adding another random oracle hash, resulting in what we call the **3-hash-Diffie-Hellman (3HDH)** OPRF.

3.1 The Functionality

The OPRF functionality is presented in Figure 2. It mostly follows the OPRF functionality used in [JKX18a]. The crucial difference is that *our functionality has an explicit notion of server’s PRF*

key, resulting in the following concrete modifications:

- Upon initialization, the functionality generates a random string k as the key for the server.
- As mentioned above, there is no command to mark a server instance as “compromised” unlocking the adversary’s ability to make offline PRF evaluation queries. Instead, anyone can attempt an offline evaluation by providing a candidate key. The correct key will give outputs consistent with the honest server, whereas each other key will correspond to an independent random function.
- The explicit PRF key can be used to index random functions. Since a man-in-the-middle adversary can replace the honest server’s key with one of its own choosing, there might be multiple independent random functions, and the functionality needs to keep track of all of them. In the OPRF of [JKX18a], the simulator cannot extract those keys, but it can detect when the same key is used twice; therefore, the functionality maintains these random functions by letting the simulator supply the “index” of the chosen function.

In our OPRF protocol, the simulator can often extract the correct key. In these cases, the index of the random function can be replaced by the actual key. However, there remain adversarial behaviors that result in a consistent random function (which the simulator can identify), but which are not consistent with any key. Although the simulator cannot extract, it turns out that neither can the adversary! More specifically, even the adversary cannot produce any string k that explains their behavior in terms of the honest PRF function. Hence, these random functions are *non-transferrable*, and we model them using the “index” approach of [JKX18a] — that is, the simulator can detect when the same index is being used and inform the functionality.

The functionality represents transferrable keys (*i.e.*, actual keys) as $(k, 1)$ and non-transferrable keys (*i.e.*, indices) as $(k, 0)$. Honest parties can make offline PRF queries *only* on a transferrable key, which models the fact that there is no key that can make the honest PRF function agree with this random function.

3.2 The 3-hash Diffie-Hellman Protocol

The 2-hash Diffie-Hellman (2HDH) OPRF protocol [JKKX17] is an oblivious protocol for evaluating the following PRF:

$$F(k, x) = H_2(x, H_1(x)^k)$$

Because k is used as an exponent, it is not feasible for the simulator to extract k . The protocol is proven secure when H_1, H_2 are random oracles, and the range of H_1 is a group in which the One-More Gap Diffie-Hellman (OMGDH) assumption [BNPS03, JKKX17] holds.

Our modification is to simply apply a random oracle to k before running 2HDH. The result is an oblivious protocol for evaluating the PRF:

$$F(k, x) = H_2(x, H_1(x)^{H_3(k)})$$

The rest of the protocol is unchanged. We call the resulting protocol **3-hash Diffie-Hellman (3HDH)** for obvious reasons. The formal description is in Figure 1.

Security. Our 3HDH protocol (Figure 1) realizes the OPRF functionality $\mathcal{F}_{\text{OPRF}}$ (Figure 2) under the $(N = Q_H + Q, Q)$ -OMGDH assumption, where Q_H is the number of the adversary $\mathcal{A}$ ’s H_1 queries, and Q is the number of PRFEval messages issued by the environment $\mathcal{Z}$. We assume w.l.o.g. that $\mathcal{A}$ never repeats a random oracle query. Below we describe the simulator.

Parameters:

- output length ℓ
- group $(\mathbb{G}, g, p)$
- hash functions $H_1 : \tilde{D} \rightarrow \mathbb{G}$, $H_2 : \tilde{D} \times \mathbb{G} \rightarrow \{0, 1\}^\ell$, and $H_3 : \{0, 1\}^\kappa \rightarrow \mathbb{Z}_p$ (modeled as random oracles)

Stealing Password Data

1. Upon receiving $(\text{Init}, \text{sid})$, S samples $k \leftarrow \{0, 1\}^\kappa$ and records (sid, k) .

Online Evaluation

1. Upon receiving $(\text{PRFEval}, \text{sid}, \text{ssid}, \text{S}, x)$, C:
 1. Samples $r \leftarrow \mathbb{Z}_p$ and records $(\text{sid}, \text{ssid}, r)$.
 2. Computes $a := H_1(x)^r$.
 3. Sends (ssid, a) to S and outputs $(\text{Prefix}, \text{sid}, \text{ssid}, a)$.
2. Upon receiving $(\text{ssid}, a \in \mathbb{G})$ from C and $(\text{PRFSendComplete}, \text{sid}, \text{ssid})$, S:
 1. Retrieves record (sid, k) and computes $b := a^{H_3(k)}$.
 2. Sends (ssid, b) to C.
3. Upon receiving $(\text{ssid}, b \in \mathbb{G})$ from S, C:
 1. Retrieves record $(\text{sid}, \text{ssid}, r)$ and computes $y := H_2(x, b^{1/r})$.
 2. Outputs $(\text{PRFReceiveComplete}, \text{sid}, \text{ssid}, y)$.

Figure 1: Protocol 3HDH

1. Sample $r_1, \dots, r_N \leftarrow \mathbb{Z}_p$, and compute $g_1 := g^{r_1}, \dots, g_N := g^{r_N}$. Initialize indices $J := 1$ (index of the current random group element g_J to be used) and $I := 1$ (the current function index).
2. On $\mathcal{A}$'s query $H_1(x)$,⁴ set $\text{DL}[x] := r_J$, $H_1(x) := g_J$, and $J := J + 1$.
3. On $(\text{PRFEval}, \text{sid}, \text{ssid}, \text{S})$ from $\mathcal{F}_{\text{OPRF}}$, record $\text{ClientSession}[\text{ssid}] := (r_J, a := g_J)$ and send $(\text{sid}, \text{ssid}, a)$ from C to S and $(\text{sid}, \text{ssid}, a)$ to $\mathcal{F}_{\text{OPRF}}$. Set $J := J + 1$.
4. On $(\text{PRFSendComplete}, \text{sid}, \text{ssid}, \text{S})$ from $\mathcal{F}_{\text{OPRF}}$, and $(\text{sid}, \text{ssid}, a^*)$ from $\mathcal{A}$ sent to S, send $(\text{sid}, \text{ssid}, a^*)$ to $\mathcal{F}_{\text{OPRF}}$, and check if **ServerKey** is defined. If not, sample $h \leftarrow \mathbb{Z}_p$ and record $\text{ServerKey} := (h, g^h)$. Either way, set $b := (a^*)^h$, and send $(\text{sid}, \text{ssid}, b)$ from C to S.
5. On $(\text{sid}, \text{ssid}, b^*)$ from $\mathcal{A}$ sent to C, retrieve $(r, a) := \text{ClientSession}[\text{ssid}]$ and compute $z := (b^*)^{1/r}$.
 - A. If there exists a previous $H_3(k^*)$ query with $b^* = a^{H_3(k^*)}$, send $(\text{PRFReceiveComplete}, \text{sid}, \text{ssid}, \text{C}, (k^*, 1))$ to $\mathcal{F}_{\text{OPRF}}$.
 - B. Otherwise, if **ServerKey** is defined, send $(\text{PRFReceiveComplete}, \text{sid}, \text{ssid}, \text{C}, \text{S})$ to $\mathcal{F}_{\text{OPRF}}$.
 - C. Otherwise, if there exists a record $z = \text{index}[i]$, send $(\text{PRFReceiveComplete}, \text{sid}, \text{ssid}, \text{C}, (i, 0))$ to $\mathcal{F}_{\text{OPRF}}$.
 - D. Otherwise, set $\text{index}[I] := z$ and send $(\text{PRFReceiveComplete}, \text{sid}, \text{ssid}, \text{C}, (I, 0))$ to $\mathcal{F}_{\text{OPRF}}$. Set $I := I + 1$.

⁴The session id is part of the input of all RO queries and is omitted.

6. On $\mathcal{A}$'s query $H_2(x, u)$, if $\text{DL}[x] = r$, set $H_2(x, u) := y$ where y is defined as follows:
 - A. If there exists a previous $H_3(k)$ query with $g^{H_3(k)} = u^{1/r}$, send $(\text{OfflineEval}, \text{sid}, (k, 1), x)$ to $\mathcal{F}_{\text{OPRF}}$ and obtain $\mathcal{F}_{\text{OPRF}}$'s response $(\text{OfflineEval}, \text{sid}, y)$.
 - B. Otherwise, if $\mathbf{S}$ is not compromised and $(h, z) := \mathbf{ServerKey}$ is defined with $z = u^{1/r}$, choose a new sub-session id ssid^* that has not appeared anywhere else in the game, send $(\text{PRFEval}, \text{sid}, \text{ssid}^*, \perp, x)$ followed by $(\text{PRFReceiveComplete}, \text{sid}, \text{ssid}^*, \text{Sim}, \mathbf{S})$ to $\mathcal{F}_{\text{OPRF}}$, and obtain $\mathcal{F}_{\text{OPRF}}$'s response $(\text{PRFReceiveComplete}, \text{sid}, \text{ssid}, y)$. If $\mathcal{F}_{\text{OPRF}}$ does not respond, abort.
 - C. Otherwise, if there exists a record $z := \text{index}[I]$ such that $z = u^{1/r}$, send $(\text{OfflineEval}, \text{sid}, (i, 0), x)$ to $\mathcal{F}_{\text{OPRF}}$ and obtain $\mathcal{F}_{\text{OPRF}}$'s response $(\text{OfflineEval}, \text{sid}, y)$.
 - D. Otherwise, set $\text{index}[I] = z$, send $(\text{OfflineEval}, \text{sid}, (I, 0), x)$ to $\mathcal{F}_{\text{OPRF}}$ and obtain $\mathcal{F}_{\text{OPRF}}$'s response $(\text{OfflineEval}, \text{sid}, y)$. Set $I := I + 1$.

If $H_2(x, u)$ is still undefined, set $H_2(x, u) \leftarrow \{0, 1\}^\ell$.

7. On $\mathcal{A}$'s query $H_3(k)$, send $(\text{TestKey}, \text{sid}, k)$ to $\mathcal{F}_{\text{OPRF}}$.
 - A. If $(h, z) := \mathbf{ServerKey}$ is defined, and $\mathcal{F}_{\text{OPRF}}$ responds with $(\text{TestKey}, \text{sid}, \mathbf{S}, k)$, set $H_3(k) := h$.
 - B. If $\mathbf{ServerKey}$ is undefined, and $\mathcal{F}_{\text{OPRF}}$ responds with $(\text{TestKey}, \text{sid}, \mathbf{S}, k)$, sample $h := H_3(k) \leftarrow \mathbb{Z}_p$ and set $\mathbf{ServerKey} := (h, g^h)$.
 - C. If $\mathcal{F}_{\text{OPRF}}$ responds with $(\text{TestKey}, \text{sid}, \mathbf{S}, \perp)$, set $H_3(k) \leftarrow \mathbb{Z}_p$.

We briefly compare our simulator and the simulator for 2HDH [JKX18a, Appendix B]:

In the interactive OPRF protocol, the client sends $a := H_1(x)^r$, and the server raises this value to the exponent $H_3(k)$, resulting in $b := H_1(x)^{rH_3(k)}$. Following the logic of 2HDH, when the adversary is playing the role of a server, the simulator is able to detect when the same exponent is used. The 2HDH simulation strategy does not give any way to extract this exponent. In our 3HDH variant, the simulator has access to all the queries made by the adversary to H_3 . Therefore, it can test all of them to see whether the server has raised the group element to that power. If so, the simulator has identified the *actual key* k (step 5A). Otherwise, the simulator is still able to associate an “index” of a random function, as in the simulator for 2HDH. Additionally, although the simulator does not know the effective exponent used by the server, no future (fresh) query to $H_3(k)$ will result in that exponent, except with negligible probability. This random function therefore is associated with a non-transferrable key (steps 5B and 5C).

Whenever an adversary queries $H_2(x, u)$, the simulator checks whether $u = H_1(x)^{H_3(k)}$ for any k that has been previously queried to H_3 . If so, then this is an offline query for (k, x) (step 6A). If not, then this may represent the adversary completing an interactive query; the simulator detects these in the same manner as in 2HDH (steps 6B–6D).

Finally, our OPRF functionality gives to the adversary a mechanism **TestKey** to check whether it has successfully obtained an honest server's key. This is necessary because the simulator will simulate an honest server using a random exponent. Then later, the adversary may learn that server's key k through the expected mechanism of the protocol. When that adversary queries $H_3(k)$, the simulator will need to detect this k as a special value, and program H_3 's output to be the exponent that it has been using (step 7A).

The security argument is very similar to that in [JKK14], so we only provide a sketch. The only case where the simulator might fail to generate a view identical to the real-world view lies in step 6B, when $\mathcal{F}_{\text{OPRF}}$ does not respond to the simulator’s `PRFReceiveComplete` message. This event can be reduced to the one-more gap Diffie-Hellman assumption: the reduction generates $g_1, \dots, g_N$ by querying the h -th power oracle, and in steps 5C and 5D, even without knowing $r_1, \dots, r_N$, the reduction can use the DDH oracle to check if $DL(a, b^*)$ is equal to some other $DL(a', (b^*)')$ (where $a', (b^*)'$ are the values that appear in a previous client sub-session) and determine the index accordingly. Similarly, in step 7A the reduction can check if $DL(g^{H_3(k)}, u) = DL(x, g_j)$, and in steps 7C and 7D it can check if $DL(g_j, u)$ is equal to some $DL(a', (b^*)')$ in a previous client sub-session. In sum, the reduction can run the simulator by replacing any computation using $r_1, \dots, r_N$ with DDH oracle queries, and when the “bad event” happens it can break the one-more gap Diffie-Hellman assumption.

4 Security Model

In this section, we detail the security modeling for fuzzy strong asymmetric PAKE. Our functionality $\mathcal{F}_{\text{safPAKE}}$ (Figure 3, Figure 4) merges the fuzzy password testing interfaces of $\mathcal{F}_{\text{faPAKE}}$ [EHOR20] and the strong asymmetric security of $\mathcal{F}_{\text{saPAKE}}$ [JKX18a]. We note that this is not the standard saPAKE functionality, and our use of a relaxed OPRF functionality (as [JKX18a] does) requires that we inherit their functionality as a base. The three changes we have made to $\mathcal{F}_{\text{saPAKE}}$ are

- We parameterize $\mathcal{F}_{\text{safPAKE}}$ to use any similarity predicate σ instead of the single definition of similarity ($\sigma(x, x') := x \stackrel{?}{=} x'$).
- We parameterize $\mathcal{F}_{\text{safPAKE}}$ to use two leakage functions ℓ_C, ℓ_S instead the default leakage function ($\ell_P(x, x') := \text{“correct guess”}$).
- We include Abdalla, Hasse, and Hesse’s strengthening of $\mathcal{F}_{\text{PAKE}}$ ’s `NewKey` interface [AHH21]. Namely, we remove the clause where if either of the parties are corrupted, the adversary may set the all parties’ keys — even if the adversary hasn’t successfully attacked the session.

We note that $\mathcal{F}_{\text{safPAKE}}$ can be parameterized with any predicate and leakage, however our protocol can support only certain predicates and leakage. We elaborate later when describing the protocol.

4.1 Leakage Function

Following previous formalizations of fuzzy PAKE [DHP⁺18, EHOR20] we provide a generalized `OfflineTestPwd` and `TestPwd` interface which can return leakage to the caller on a correct guess. Dupont et al. [DHP⁺18] define three leakage functions, based on what is leaked in the case of far/medium/close password guesses (under some metric).

We handle leakage slightly differently. We allow the functionality to be parameterized by two different leakage functions: One for the client ℓ_C and one for the server ℓ_S . Specifically, if the password test is against party P , we use leakage function ℓ_P . In the case where the passwords are similar, we give the adversary $\ell_P(pw, pw')$ in an all-or-nothing situation. In any other case, the functionality simply replies “wrong password.” Our final safPAKE protocol supports a range of σ predicates, and leakage $\ell_S(pw, pw') = pw$, $\ell_C(pw, pw') = \text{“correct guess”}$. That is, the client learns the “correct” password and key agreement succeeds when the two passwords are similar while from the server’s perspective, the key agreement simply succeeds or doesn’t.

Functionality $\mathcal{F}_{\text{OPRF}}$

Parameters: output length ℓ

Storage: four maps: `key`, `output`, `tx`, and `session`

Initialization

Upon receiving $(\text{Init}, \text{sid})$ from honest P :

1. Sample $k \leftarrow \{0, 1\}^\kappa$.
2. Record $\text{key}[P, \text{sid}] := k$.
3. Send $(\text{Init}, \text{sid}, k)$ to P .

Offline Evaluation

Upon receiving $(\text{OfflineEval}, \text{sid}, (k, \text{trans}), x)$ from P :

1. If P is honest, overwrite $\text{trans} := 1$.
2. If $\text{output}[k, \text{trans}, x]$ is undefined, sample $\text{output}[k, \text{trans}, x] \leftarrow \{0, 1\}^\ell$.
3. Send $(\text{OfflineEval}, \text{sid}, \text{output}[k, \text{trans}, x])$ to P .

Upon receiving $(\text{TestKey}, \text{sid}, k)$ from $\mathcal{A}^*$:

1. If $\text{key}[P, \text{sid}] = k$ for some party P , then send $(\text{TestKey}, \text{sid}, k, P)$ to $\mathcal{A}^*$.
2. Otherwise send $(\text{TestKey}, \text{sid}, k, \perp)$ to $\mathcal{A}^*$.

Online Evaluation

Upon receiving $(\text{PRFEval}, \text{sid}, \text{ssid}, S, x)$ from $P \in \{C, \mathcal{A}^*\}$:

1. Send $(\text{PRFEval}, \text{sid}, \text{ssid}, P, S)$ to $\mathcal{A}^*$.
2. Await response $(\text{sid}, \text{ssid}, \text{prfx})$ from $\mathcal{A}^*$, and if $P = C$, ignore this command if $(\text{Prefix}, \text{sid}, \text{ssid}, \text{prfx})$ was delivered to C or S previously.
3. Otherwise, set $\text{session}[(\text{ssid}, P)] := (x, \text{prfx})$ and send $(\text{Prefix}, \text{sid}, \text{ssid}, \text{prfx})$ to P .

Upon receiving $(\text{PRFSendComplete}, \text{sid}, \text{ssid})$ from S :

1. Send $(\text{PRFSendComplete}, \text{sid}, \text{ssid}, S)$ to $\mathcal{A}^*$.
2. Await response $(\text{sid}, \text{ssid}, \text{prfx}')$ from $\mathcal{A}^*$, and send $(\text{Prefix}, \text{sid}, \text{ssid}, \text{prfx}')$ to S .
3. If $(x, \text{prfx}) := \text{session}[(\text{ssid}, C)]$ is defined with $\text{prfx} = \text{prfx}'$, store $\text{session}[(\text{ssid}, C)] := (x, \text{OK})$.
4. Otherwise set $\text{tx}[S] := \text{tx}[S] + 1$.

Upon receiving $(\text{PRFReceiveComplete}, \text{sid}, \text{ssid}, P, i)$ from $\mathcal{A}^*$:

1. If either $(x, \text{prfx}) := \text{session}[(\text{ssid}, P)]$ is undefined or $(i = S, \text{tx}[S] = 0, \text{and } \text{prfx} \neq \text{OK})$, then ignore this command.
2. If $i = S$, set $(k, \text{trans}) := (\text{key}[S, \text{sid}], 1)$. Otherwise, set $(k, \text{trans}) := i$.
3. If $\text{output}[k, \text{trans}, x]$ is undefined, then sample $\text{output}[k, \text{trans}, x] \leftarrow \{0, 1\}^\ell$.
4. Give $(\text{PRFReceiveComplete}, \text{sid}, \text{ssid}, \text{output}[k, \text{trans}, x])$ to P .
5. If $i = S$ and $\text{prfx} \neq \text{OK}$ then set $\text{tx}[S] := \text{tx}[S] - 1$.

Figure 2: Ideal functionality $\mathcal{F}_{\text{OPRF}}$

4.2 Functionality

In our functionality, we consider an asymmetric interaction where one party takes on the role of the client C who has a password as input pw' and the other takes on the role of the server S who holds a “password file” `file` which is a digest of some password pw . The goal of the interaction is to arrive at a shared key exactly when $\sigma(pw, pw') \neq \perp$.

Since the parties are assumed to supply low-entropy passwords, an integral security property of PAKE lies in allowing the adversary *exactly one password guess* per session. $\mathcal{F}_{\text{saffPAKE}}$ provides this interface in the form of the **TestPwd** query. This query can be sent once per subsession and provides the adversary a single password guess. We note that a single password guess will encompass a potentially large “guessing surface” of passwords that satisfy the similarity predicate with the target password. This is inherent to any PAKE protocol that supports fuzzy matches, and it is important to note that this is different from allowing an adversary an arbitrary “guessing surface” of the same size. An adversary can guess only the “similarity neighborhood” Σ_{pw^*} of one password pw^* . This can be seen in the **TestPwd** interface which only allows a single password as input.

Functionality $\mathcal{F}_{\text{safPAKE}}$

Parameters:

- security parameter κ
 - client C , server S , and ideal adversary $\mathcal{A}^*$
- symmetric similarity predicate $\sigma : \tilde{D} \times \tilde{D} \rightarrow \{\perp\} \cup \{0, 1\}^\kappa$
 - leakage functions $\ell_C, \ell_S : \tilde{D} \times \tilde{D} \rightarrow \{0, 1\}^*$

Storage:

- Five maps: `fileStatus`, `file`, `sessionStatus`, `session`, and `delayedTests`
- A server compromise flag `flag`

Password Registration

Upon receiving `(StorePwdFile, sid, C, pw)` from S , if `file[(sid, C, S)]` is undefined, set `file[(sid, C, S)] := pw`, set `fileStatus[(sid, C, S)] := uncompromised`, and set `flag := uncompromised`.

Stealing Password Data

Upon receiving `(StealPwdFile, sid)` from $\mathcal{A}^*$:

1. If `file[(sid, C, S)]` is undefined, return “no password file” to $\mathcal{A}^*$.
2. Otherwise:
 1. If `fileStatus[(sid, C, S)] = uncompromised`, set `fileStatus[(sid, C, S)] := stolen`.
 2. Return “password file stolen” to $\mathcal{A}^*$.

Upon receiving `(OfflineTestPwd, sid, pw*)` from $\mathcal{A}^*$:

1. If `fileStatus[(sid, C, S)] = stolen`:
 1. Retrieve `pw := file[(sid, C, S)]`.
 2. If $\sigma(pw, pw*) \neq \perp$: return $\ell_S(pw, pw*)$ to $\mathcal{A}^*$ and set `flag := compromised`.
 3. Otherwise, return “wrong guess” to $\mathcal{A}^*$.

Password Authentication

Upon receiving `(ClientSession, sid, ssid, S, pw')` from C :

1. Send `(ClientSession, sid, ssid, C, S)` to $\mathcal{A}^*$.
2. If `sessionStatus[(sid, ssid, C, S)]` is undefined, set `session[(sid, ssid, C, S)] := pw'` and set `sessionStatus[(sid, ssid, C, S)] := fresh`.

Upon receiving `(ServerSession, sid, ssid)` from S :

1. If `file[(sid, C, S)]` is undefined, ignore this query.
2. Otherwise, retrieve `pw := file[(sid, C, S)]` and send `(ServerSession, sid, ssid, C, S)` to $\mathcal{A}^*$.
3. If `sessionStatus[(sid, ssid, S, C)]` is undefined, set `session[(sid, ssid, S, C)] := pw` and set `sessionStatus[(sid, ssid, S, C)] := fresh`.

Figure 3: Ideal functionality safPAKE (part 1). The framed text are generalizations of $\mathcal{F}_{\text{safPAKE}}$ [JKX18a].

Active Session Attacks

Upon receiving (Interrupt, sid, ssid, S) from $\mathcal{A}^*$:

1. If $\text{sessionStatus}[(\text{sid}, \text{ssid}, S, C)] = \text{fresh}$, set $\text{sessionStatus}[(\text{sid}, \text{ssid}, S, C)] := \text{interrupted}$ and set $\text{delayedTests}[\text{ssid}] := 1$.

Upon receiving (TestPwd, sid, ssid, P, pw^*) from $\mathcal{A}^*$:

1. If $\text{sessionStatus}[(\text{sid}, \text{ssid}, P, P')]$ is undefined, ignore this query.
2. Otherwise, retrieve $\widetilde{pw} := \text{session}[(\text{sid}, \text{ssid}, P, P')]$.
3. Let $s := \sigma(\widetilde{pw}, pw^*)$, $l := \ell_S(\widetilde{pw}, pw^*)$ if $P = S$ and let $s := \sigma(pw^*, \widetilde{pw})$, $l := \ell_C(pw^*, \widetilde{pw})$ if $P = C$.
4. If $\text{sessionStatus}[(\text{sid}, \text{ssid}, P, P')] = \text{fresh}$:
 1. If $s \neq \perp$, return l to $\mathcal{A}^*$ and set $\text{sessionStatus}[(\text{sid}, \text{ssid}, P, P')] := \text{compromised}$.
 2. Otherwise, set $\text{sessionStatus}[(\text{sid}, \text{ssid}, P, P')] := \text{interrupted}$ and return “wrong guess” to $\mathcal{A}^*$.
5. If $P = S$ and $\text{delayedTests}[\text{ssid}] = 1$: set $\text{delayedTests}[\text{ssid}] := 0$, if $s \neq \perp$, return l to $\mathcal{A}^*$, otherwise return “wrong guess” to $\mathcal{A}^*$.
6. In both cases, if $P = S$ and $s \neq \perp$, then set $\text{flag} := \text{compromised}$.

Upon receiving (Impersonate, sid, ssid) from $\mathcal{A}^*$:

1. If $\text{sessionStatus}[(\text{sid}, \text{ssid}, C, S)]$ is undefined or if $\text{fileStatus}[(\text{sid}, C, S)] \neq \text{stolen}$, ignore this query.
2. Otherwise, retrieve $pw := \text{file}[(\text{sid}, C, S)]$ and $pw' := \text{session}[(\text{sid}, \text{ssid}, C, S)]$.
 1. If $\text{sessionStatus}[(\text{sid}, \text{ssid}, C, S)]$ is fresh: if $\sigma(pw, pw') \neq \perp$, set $\text{sessionStatus}[(\text{sid}, \text{ssid}, C, S)] := \text{compromised}$ and return “correct guess” to $\mathcal{A}^*$; otherwise, set $\text{sessionStatus}[(\text{sid}, \text{ssid}, C, S)] := \text{interrupted}$ and return “wrong guess” to $\mathcal{A}^*$.

Key Generation and Authentication

Upon receiving (NewKey, sid, ssid, P, k^*) from $\mathcal{A}^*$ where $|k^*| = \kappa$:

1. If $\text{sessionStatus}[(\text{sid}, \text{ssid}, P, P')]$ is defined, but is not completed:
 1. Let $s := \sigma(\text{sessionStatus}[(\text{sid}, \text{ssid}, S, C)], \text{sessionStatus}[(\text{sid}, \text{ssid}, C, S)])$
 2. If the record is compromised, or if ($P = S$, the record is interrupted, and $\text{flag} = \text{compromised}$), set $k := k^*$.
 3. Else, if the record is fresh, $(\text{sid}, \text{ssid}, k')$ was sent to P' , $s \neq \perp$, and at the time $\text{sessionStatus}[(\text{sid}, \text{ssid}, P', P)]$ was fresh, set $k := k'$.
 4. Otherwise, sample $k \leftarrow \{0, 1\}^\kappa$.
2. Finally, set $\text{sessionStatus}[(\text{sid}, \text{ssid}, P, P')] := \text{completed}$ and send $(\text{sid}, \text{ssid}, k)$ to P.

Upon receiving (TestAbort, sid, ssid, P) from $\mathcal{A}^*$:

1. If $\text{sessionStatus}[(\text{sid}, \text{ssid}, P, P')]$ is defined, but is not completed:
 1. Let $s := \sigma(\text{sessionStatus}[(\text{sid}, \text{ssid}, S, C)], \text{sessionStatus}[(\text{sid}, \text{ssid}, C, S)])$
 2. If $\text{session}[(\text{sid}, \text{ssid}, P, P')]$ is fresh and $s \neq \perp$, send “success” to $\mathcal{A}^*$.
 3. Else, send “abort” to $\mathcal{A}^*$ and (Abort, sid, ssid) to P.
 4. In both cases, set $\text{sessionStatus}[(\text{sid}, \text{ssid}, P, P')] := \text{completed}$.

Figure 4: Ideal functionality safPAKE (part 2). The framed text are generalizations of $\mathcal{F}_{\text{safPAKE}}$ [JKX18a].

In (strong) asymmetric PAKE we must additionally account for the adversary learning the server’s long-term secrets. To accomplish this, the adversary is capable of sending a corruption query `StealPwdFile` to the server and receiving the server’s password file `file`. We distinguish such a query from the static *Byzantine* corruptions by not allowing the adversary to control the server or to modify the server’s long-term storage. To ensure that precomputation does not allow the adversary to “instantly” learn passwords upon compromise, the `OfflineTestPwd` interface is only available after compromise, but allows the adversary an unlimited number of guesses without online interaction. This models the adversary’s ability to perform a dictionary attack against the server’s file, which is inevitable since the adversary learns enough to simulate login attempts against the server’s password file. Aside from password guesses, the adversary may now impersonate the server through the `Impersonate` query which allows the server to use the password file `file` to connect with a client. No similar impersonation interface exists for the client. This implies that the server must identify the password underlying the password file to impersonate a client to the honest server.

5 Protocol Overview

In this section, we build up the intuition for our `safPAKE` protocol, step by step.

Step 0: JKC Compiler Jarecki, Krawczyk, and Xu, in their seminal paper on strong asymmetric PAKE (saPAKE) [JKX18a], provided a compiler from plain aPAKE to saPAKE. The server holds a PRF key k and the “aPAKE digest” of $rw = F(k, pw)$, where F is a PRF. To authenticate, the client with input pw' learns $F(k, pw')$ via an oblivious PRF (OPRF) protocol. The parties then perform aPAKE where the client uses $rw' := F(k, pw')$ as input, and the server uses its digest of rw . Authentication succeeds if $rw' = rw$, which happens only when $pw' = pw$.

Step 1: Run many instances. The intuitive starting point for `safPAKE` is to run t parallel instances of saPAKE, where t is the number of acceptable mistyped passwords. The client, who holds a password guess pw' , should use it as input to all instances. The server holds an saPAKE password file for each possible mistyping of the correct password pw . In this way, pw' is compared against every mistyping of pw .

This naïve `safPAKE` protocol has several deficiencies, which we address next. First, both parties can make t guesses of the other’s password in each authentication attempt. These t guesses can be completely unrelated — i.e., they need not correspond to a “typo neighborhood” around a single correct password. Second, the protocol is a factor t more expensive than saPAKE.

Step 2: Reuse OPRF. The client is supposed to use the same pw' guess to each of the t saPAKE instances. It is easy to enforce this by using just a single OPRF instance of t of them. With just one OPRF instance, the client learns $F(k, pw')$ for its single password guess pw' . Now we need a way to check whether this OPRF output is consistent with one of the allowed mistypings of pw .

A natural way to do this is for the server to hold and send “correction ciphertexts” to the client. For every allowed mistyping $\widetilde{pw}$ of the correct password pw , there is a ciphertext that encrypts $rw = F(k, pw)$ under the key $F(k, \widetilde{pw})$. Suppose the client obtains $F(k, pw')$ from an OPRF instance, and also holds these ciphertexts. Intuitively, if the client’s OPRF input is a valid mistyping of pw , then she can decrypt one of these ciphertexts and obtain $rw = F(k, pw)$. If her OPRF input is not a valid mistyping of pw then the ciphertexts are meaningless to her. Then the parties can perform a single aPAKE instance, where the server holds a digest of rw .

Step 3: Check all the correction ciphertexts. In step 2, we successfully restricted the client to a single password guess pw' . However, the server can still make t completely unrelated guesses of the client's password. It can encrypt rw , t times, under the keys $F(k, pw_1), \dots, F(k, pw_t)$, where the pw_i 's are completely unrelated instead of being the set of valid mistypings of a single password. As the server must be able to authenticate the client against t possible inputs, the server will always be allowed t guesses. However, in the best case scenario, the server must have all of these guesses (structurally) be known by the client upon connection.

We fix this problem with the following two ideas: First, the correction ciphertexts are encryptions of (k, pw) ⁵ instead of encryptions of rw . Second, the correction ciphertexts should be computed as a *deterministic function* of k and pw . Specifically, the randomness used for encryption should be derived from k and pw .

With these two ideas in place, whenever the client successfully decrypts a correction ciphertext, she obtains (k, pw) . From (k, pw) she can not only compute $rw = F(k, pw)$ herself, but also compute the *entire set of correction ciphertexts*. Importantly, she can compute them knowing the set of valid mistypings of pw .⁶ If she realizes that the server did not send this *exact* collection of ciphertexts, she will abort the interaction.

Intuitively, if the server sends correction ciphertexts that do not correctly correspond to the valid set of mistypings of a single password, then the client will *always* abort the protocol — either because the client couldn't open any ciphertext, or because the client opened a ciphertext but then detected that the set of ciphertexts was incorrect.

Attentive readers will notice that there is some circularity in the new correction ciphertexts, where its plaintext (k, pw) is hidden, because the encryption key $F(k, \widetilde{pw})$ is random, because the PRF seed k is hidden. We must indeed be careful about such circularity! Fortunately, saPAKE protocols like OPAQUE already require a very strong OPRF where $F(k, x)$ can look random even to someone who knows k — provided that they haven't queried the PRF at $F(k, x)$ yet. Such a strong [O]PRF is enough to resolve the apparent circularity.

Step 4: Compress communication with PIR. As a result of the previous step, we have a saPAKE protocol where both the client and server can make only a single password guess. However, the protocol's communication cost still scales with t , the number of valid mistypings. This is due to the t correction ciphertexts, which the server must send to the client.

We can compress this communication by using PIR. In our saPAKE protocol, there is only one ciphertext that the client can decrypt using its key $F(k, pw')$. If the client can find a way to “identify” which ciphertext to decrypt, then she can use (single-server) **keyword PIR** to fetch it from the server with low communication.

In more detail, let us interpret the output of $F(k, \widetilde{pw})$ in two pieces: $\text{tag}_{\widetilde{pw}} \| \text{ek}_{\widetilde{pw}} := F(k, \widetilde{pw})$, where ek is an encryption key and tag is a random identifier. The server can prepare a key-value mapping that maps $\text{tag}_{\widetilde{pw}}$ to the correction ciphertext $\text{Enc}(\text{ek}_{\widetilde{pw}}; (k, pw))$.⁷ The client learns $F(k, pw') = \text{tag}_{pw'} \| \text{ek}_{pw'}$, and then initiates a keyword PIR protocol on input $\text{tag}_{pw'}$. If this is the tag corresponding to some ciphertext, then the client will obtain that ciphertext and be able to decrypt it as usual.

But whenever the client successfully decrypts the ciphertext, it must then recompute the *entire set* of correction ciphertexts and compare to what the server has sent. However, now the server

⁵The server actually provides a way for the client to reconstruct pw instead of providing pw directly, but for simplicity in this section, we write the plaintexts as containing pw .

⁶This requires the password typo policy to be public.

⁷For simplicity, we are ignoring the encryption randomness which also should be derived from the output of $F(k, \widetilde{pw})$.

is not actually sending the entire collection of ciphertexts, but is only running a PIR protocol. We resolve this problem by observing that typical PIR protocols can have deterministic server behavior (because PIR has no privacy guarantees for the server). So the client can recompute (in her head) the entire collection of correction ciphertexts, and check whether the server *followed the deterministic PIR protocol with this collection as its input*.

The consistency check in Step 3 (the client recomputes all the correction ciphertexts and compares to what the server previously sent) is in fact an instantiation of this deterministic-PIR consistency check, because sending the entire ciphertext database is a trivial PIR protocol.

5.1 Similarity Predicates

As we briefly introduced in the previous section, the client needs to know the set of valid mistypings of her input pw' in order to verify the server's computation. We define this set in terms of a **similarity predicate** σ . In traditional PAKE, we concern ourselves with a password dictionary D from which the parties can choose their inputs from. As we now must consider errors, we use an expanded dictionary $\tilde{D}$ in our definitions.

Definition 2 (Similarity Predicate). *A **similarity predicate** is an efficiently computable function $\sigma : \tilde{D} \times \tilde{D} \rightarrow \{\perp\} \cup \{0, 1\}^\kappa$ along with an efficiently computable **reconstruction function** $\rho : \tilde{D} \times \{0, 1\}^\kappa \rightarrow \{\perp\} \cup \tilde{D}$ such that*

- *correctness: $\forall x, x' \in \tilde{D}$ such that $y := \sigma(x, x') : y \neq \perp \implies \rho(x', y) = x$.*
- *reconstruction bounded: $\forall x \in \tilde{D}, y \in \{0, 1\}^\kappa$ such that $z := \rho(x, y) : z \neq \perp \implies x \in \Sigma_z$ where $\Sigma_z := \{x' \mid \sigma(z, x') \neq \perp\}$.*

In other words, x is said to be “similar” to x' if and only if x can be “reconstructed” from x' : $\rho(x', \sigma(x, x')) = x$. The set Σ_x is then said to be the set of all values “similar” to x .

We require four additional properties of σ :

- *self similarity: $\forall x \in \tilde{D}, \sigma(x, x) \neq \perp$.*
- *input independent size: $\forall x \in \tilde{D}, \Sigma_x$ is of some fixed cardinality n .*
- *efficient enumerability: $\forall x \in \tilde{D}, \Sigma_x$ can be enumerated in polynomial time.*
- *predictable hints: $\forall x \in \tilde{D}, \sigma(x, \Sigma_x) = \{1, 2, 3, \dots, n\}$.*

We assume that $\tilde{D}$, and n are part of σ 's description. It is important to note that our definition of predictable hints may seem stronger than necessary for our proof. Indeed, the set does not need to be exactly $\{1, 2, 3, \dots, n\}$, but rather just needs to be some fixed set. We use $\{1, 2, 3, \dots, n\}$, without loss of generality, to not bog down notation when we use the set's lexicographic ordering as an indexing set. We also conjecture that given a PIR protocol which hides the size of the server's database, we can drop the *input independent size* property for a weaker property which only requires an upper bound on the size of Σ_x .

The general definition covers many fuzzy distance functions such as repetition mistakes, capitalization errors, and hamming distance for fixed length passwords. This is a similar notion to the implicit requirements for the aPAKE to fuzzy aPAKE compiler due to Erwig et al. [EHOR20]. Additionally, for any *symmetric* similarity predicate which is also efficiently enumerable, we may generically implement $\sigma(x, x')$ by outputting the index of x in $\Sigma_{x'}$ (following some known ordering) and $\rho(x', y)$ can then output the y th element of $\Sigma_{x'}$.

6 A Strong Asymmetric Fuzzy PAKE Protocol

In this section we describe a compiler from an OPRF, PIR protocol, and an aPAKE protocol to a safPAKE protocol. The actual protocol isn't far from the intuition we have built so far and we present a graphical representation of the protocol in [Figure 5](#).

Registration

In the registration phase, the server begins by interacting with the OPRF functionality to receive the user-specific key k and computes its salted password rw from $F(k, pw)$. The server then initializes the aPAKE server storage using rw and constructs its local PIR database centered around pw . This is accomplished by mapping password tags $f_{pw_i}.tag$ to encryptions of the corresponding reconstruction hints $d_{pw_i} := \sigma(pw, pw_i)$.

1. Upon receiving (StorePwdFile, sid, C, pw), S sends (Init, sid) to $\mathcal{F}_{\text{OPRF}}$ and receives (Init, sid, k).
2. S then sends (OfflineEval, sid, (k, 1), $\widetilde{pw}$) to $\mathcal{F}_{\text{OPRF}}$ for each $\widetilde{pw} \in \Sigma_{pw} := \{\widetilde{pw} \in \widetilde{D} \mid \sigma(pw, \widetilde{pw}) \neq \perp\}$.
3. Upon receiving (OfflineEval, sid, $f_{\widetilde{pw}}$), S parses $f_{\widetilde{pw}}$ as four equilength strings $f_{\widetilde{pw}}.tag \parallel f_{\widetilde{pw}}.ek \parallel f_{\widetilde{pw}}.rs \parallel f_{\widetilde{pw}}.rw$ each of length κ . We recall that as σ provides predictable hints, we may use these hints as an ordering for passwords $\Sigma_{pw} = \{pw_1, pw_2, \dots, pw_n\}$.
4. S then generates keyword PIR database $\text{DB} := \{f_{pw_i}.tag \mapsto \mathcal{E}.\text{Enc}(f_{pw_i}.ek; (k, d_{pw_i} := \sigma(pw, pw_i)); f_{pw_i}.rs)\}$ for $i \in \{1, 2, \dots, n\}$.
5. Finally, S sends (StorePwdFile, sid, C, $f_{pw}.rw$) to $\mathcal{F}_{\text{aPAKE}}$ and stores $\text{file}[\text{sid}] := (\text{DB}, k)$.

Server Compromise

1. Upon receiving (StealPwdFile, sid) from $\mathcal{A}$, S retrieves $\text{file}[\text{sid}]$ and sends it to $\mathcal{A}$.

Login

During login, the client learns a PIR tag associated with their password pw' by evaluating the OPRF with the server. The client and server then perform a PIR exchange on the client's tag. Once the client receives the response, the client decrypts the reconstruction hint d' and reconstructs the server's password $pw := \rho(pw', d')$. The client then verifies the server's messages and runs an aPAKE subsession with the server on the salted password rw computed from $F(k, pw)$.

1. Upon receiving (ClientSession, sid, ssid, S, pw'), C sends (PRFEval, sid, ssid, S, pw') to $\mathcal{F}_{\text{OPRF}}$ receiving (Prefix, sid, ssid, $prfx_C$).
2. Upon receiving (ServerSession, sid, ssid), S retrieves $(\text{DB}, k) := \text{file}[\text{sid}]$ and sends (PRFSendComplete, sid, ssid) to $\mathcal{F}_{\text{OPRF}}$ receiving (Prefix, sid, ssid, $prfx_S$).
3. Upon receiving (PRFReceiveComplete, sid, ssid, f) from $\mathcal{F}_{\text{OPRF}}$ C parses f as four equilength strings $f.tag \parallel f.ek \parallel f.rs \parallel f.rw$ then C computes $(\text{state}, \text{req}) \leftarrow \text{PIR.Request}(f.tag)$ and sends (sid, ssid, req) to S.
4. Upon receiving (sid, ssid, req) from C, S computes $\text{resp} := \text{PIR.Respond}(\text{req}, \text{DB})$, queries $\mathcal{F}_{\text{RO}}$ on input DB receiving $h \in \{0, 1\}^\kappa$ and sends (sid, ssid, (resp, h)) to C.
5. Upon receiving (sid, ssid, (resp, h)) from S, C computes $v := \text{PIR.Finish}(\text{resp}, \text{state})$.
 1. C computes $z := \mathcal{E}.\text{Dec}(f.ek, v)$ and checks if z parses as $(k' \in \{0, 1\}^\kappa, d' \in \{0, 1\}^\kappa)$.
 2. If so, C then sets $pw := \rho(pw', d')$.
 3. C sends (OfflineEval, sid, (k', 1), pw_i) to $\mathcal{F}_{\text{OPRF}}$ for each $pw_i \in \Sigma_{pw}$ and catalogs the responses f_{pw_i} .

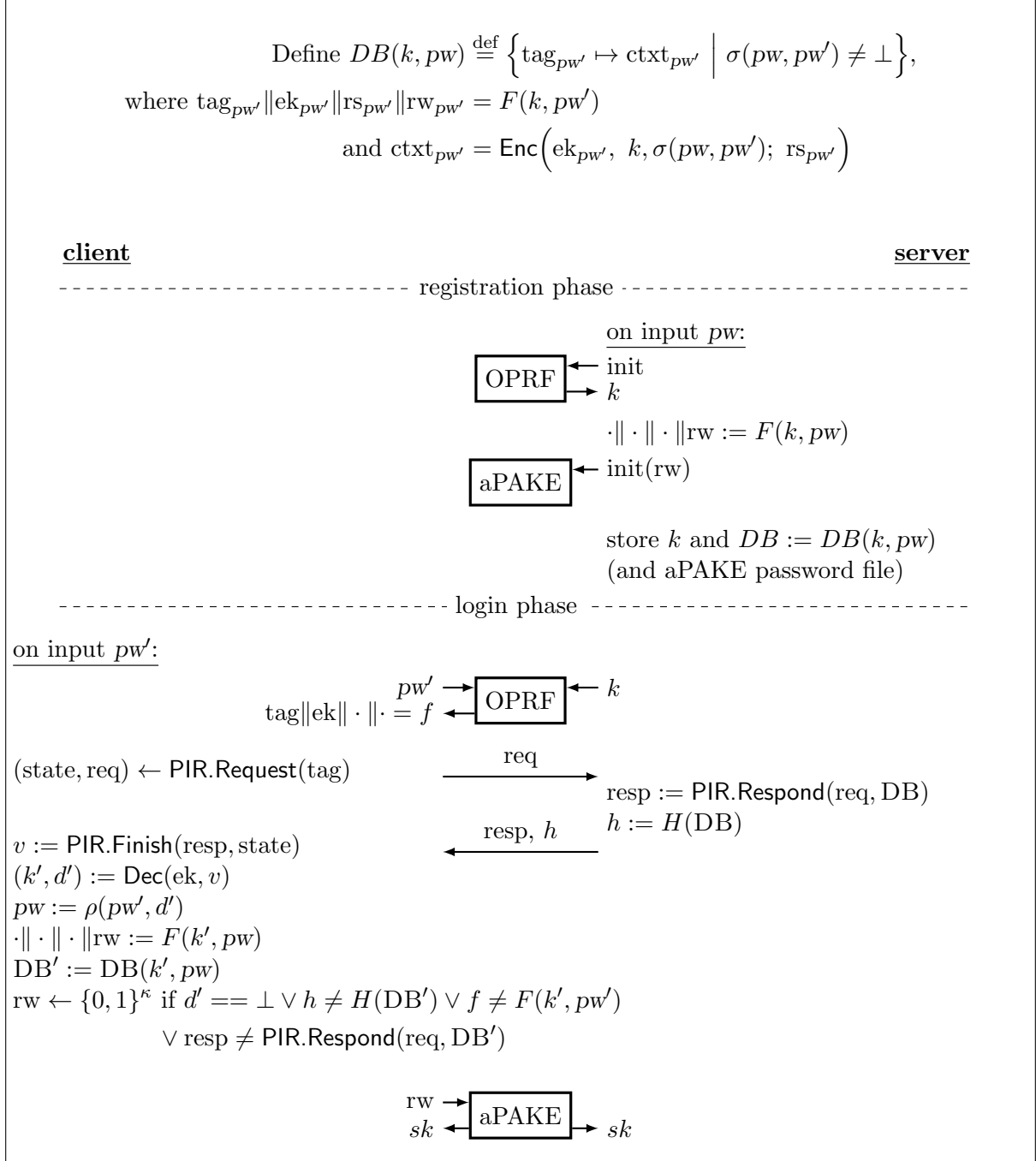


Figure 5: Graphical representation of our protocol. See text for omitted details.

4. C generates PIR database DB' using k' and the f_{pw_i} then computes $\text{resp}' := \text{PIR.Respond}(\text{req}, DB')$.
5. C queries $\mathcal{F}_{\text{RO}}$ on input DB' receiving $h' \in \{0, 1\}^\kappa$.
6. C checks if $\text{resp}' = \text{resp}$ and $h' = h$.
7. If this check fails, parsing z failed, or $f_{pw'} \neq f$ C sets $f_{pw}.\text{rw} \leftarrow \{0, 1\}^\kappa$.
8. C sends $(\text{ClientSession}, \text{sid}, \text{ssid}', S, f_{pw}.\text{rw})$ to $\mathcal{F}_{\text{aPAKE}}$ where $\text{ssid}' = \text{ssid} \parallel \text{prfx}_C \parallel \text{req} \parallel \text{resp} \parallel h$.

6. S sends $(\text{ServerSession}, \text{sid}, \text{ssid}')$ to $\mathcal{F}_{\text{aPAKE}}$ where $\text{ssid}' = \text{ssid} \parallel \text{prfx}_S \parallel \text{req} \parallel \text{resp} \parallel h$.
7. Either party, upon receiving $(\text{sid}, \text{ssid}', \text{sk})$ or $(\text{Abort}, \text{sid}, \text{ssid}')$ from $\mathcal{F}_{\text{aPAKE}}$, outputs the message received.

7 Security Analysis

We will now show that our protocol is a secure realization of the $\mathcal{F}_{\text{safPAKE}}$ functionality.

Theorem 3. *The protocol in Section 6 securely realizes the safPAKE functionality $\mathcal{F}_{\text{safPAKE}}$ (Figure 3 and Figure 4) in the $(\mathcal{F}_{\text{RO}}, \mathcal{F}_{\text{aPAKE}}, \mathcal{F}_{\text{OPRF}})$ -hybrid model and the static Byzantine corruption setting, for any authenticated encryption scheme (Definition 1) $\mathcal{E}$; efficiently enumerable, input-independent-sized, self-similar, similarity predicate with predictable hints (Section 5.1) σ ; secure keyword private information retrieval scheme (PIR.Request , PIR.Respond , PIR.Finish) (Section 2.1); and leakage functions $\ell_S(\text{pw}, \text{pw}') = \text{pw}$, $\ell_C(\text{pw}, \text{pw}') = \text{"correct password"}$.*

7.1 Simulator

We construct the following simulator Sim for any PPT environment $\mathcal{Z}$. As standard in UC, we assume that the real adversary $\mathcal{A}$ is “dummy”, i.e., it merely passes messages to and from $\mathcal{Z}$. Without loss of generality, we also assume that all $\mathcal{F}_{\text{RO}}$ queries are made via $\mathcal{A}$, i.e., $\mathcal{Z}$ does not make these queries on its own. In the following, the session ID is always included as part of a random oracle input and is omitted (i.e., $H(\text{sid}, x)$ is simplified to $H(x)$).

Generating the Password File Upon receiving the first server message for a given sid , Sim generates a database on uniform values as follows. We recall that for any input $x \in \tilde{D}$, the set of all similar inputs $\Sigma_x = \{\tilde{x} \mid \sigma(x, \tilde{x}) \neq \perp\}$ is of some fixed size n .

1. Sample n uniform strings $f_1, \dots, f_n \leftarrow \{0, 1\}^{3\kappa}$, sample two strings $k \leftarrow \{0, 1\}^\kappa$, $\text{rw} \leftarrow \{0, 1\}^\kappa$, and store these values $\text{params}[\text{sid}] := \{f_1, \dots, f_n\}$, $\text{rw}[\text{sid}] := \text{rw}$.
2. For $i \in \{1, 2, 3, \dots, n\}$, parse $f_i = f_i.\text{tag} \parallel f_i.\text{ek} \parallel f_i.\text{rs}$ each with length κ . If any tag, ek, or rs collide, abort. Then generate the database $\text{database}[\text{sid}] := \{f_i.\text{tag} \mapsto \mathcal{E}.\text{Enc}(f_i.\text{ek}; (k, i); f_i.\text{rs})\}$.
3. Finally, store $\text{file}[\text{sid}] := (\text{database}[\text{sid}], k)$ and set $\text{password}[\text{sid}] := \perp$.

The difference between what the simulator does and a server’s honest run of the protocol is that the honest server would receive a random OPRF key k from $\mathcal{F}_{\text{OPRF}}$, and would compute $f_i \parallel f_i.\text{rw} := F_{k,1}(\text{pw}_i)$ and $\text{rw} := F_{k,1}(\text{pw}).\text{rw}$ using OfflineTestPwd queries to $\mathcal{F}_{\text{OPRF}}$.

Programming the Password File In many parts of the simulator, when $\mathcal{A}$ successfully guesses the session’s password through OfflineTestPwd or through TestPwd , Sim needs to equivocate or “explain” the password file it previously generated. To do so, we define a function the simulator can call to, $\text{ProgramDatabase}(\text{sid}, \text{pw}, \text{rw}, k, \{f_1, \dots, f_n\})$, as follows:

1. Compute Σ_{pw} . Recall that by the predictable hints property of σ , we may number all elements $\tilde{\text{pw}} \in \Sigma_{\text{pw}}$ with $\tilde{\text{pw}}$ being assigned $\sigma(\text{pw}, \tilde{\text{pw}})$.
2. For each $\text{pw}_i \in \Sigma_{\text{pw}}$, if $F_{k,1}(\text{pw}_i)$ is undefined and $\text{pw}_i \neq \text{pw}$, sample $\text{rw}_i \leftarrow \{0, 1\}^\kappa$ and set $F_{k,1}(\text{pw}_i) := f_i \parallel \text{rw}_i$. If any rw_i collide, abort.
3. If $F_{k,1}(\text{pw})$ is undefined, set $F_{k,1}(\text{pw}) := f_j \parallel \text{rw}$ where j is the index of pw in Σ_{pw} .

Stealing the Password File and Offline Queries

4. Initialize $\text{tested}[\text{sid}] := \emptyset$.
5. Upon receiving $(\text{StealPwdFile}, \text{sid})$ from $\mathcal{A}$ sent to $\mathcal{F}_{\text{aPAKE}}$, and $(\text{StealPwdFile}, \text{sid})$ from $\mathcal{A}$ sent to $\mathcal{S}$:
 - A. Send $(\text{StealPwdFile}, \text{sid})$ to $\mathcal{F}_{\text{safPAKE}}$.
 - B. If $\mathcal{F}_{\text{safPAKE}}$ returns “password file stolen”, then
 - I. Mark $\mathcal{S}$ compromised.
 - II. For $\text{rw} := \text{rw}[\text{sid}]$, if $\text{rw} \in \text{tested}[\text{sid}]$, then return rw to $\mathcal{A}$ from $\mathcal{F}_{\text{aPAKE}}$. Otherwise, return “password file stolen” to $\mathcal{A}$ from $\mathcal{F}_{\text{aPAKE}}$.
 - III. Return $\text{file}[\text{sid}]$ to $\mathcal{A}$ from $\mathcal{S}$.
 - C. Otherwise, return “no password file” to $\mathcal{A}$ as a message from $\mathcal{F}_{\text{aPAKE}}$.
6. Upon receiving $(\text{OfflineEval}, \text{sid}, (k^*, \text{trans}^*), x)$ from $\mathcal{A}$ sent to $\mathcal{F}_{\text{OPRF}}$:
 - A. If $\mathcal{S}$ is marked compromised, $k = k^*$, and $\text{trans} = 1$ where $(\text{DB}, k) := \text{file}[\text{sid}]$, send $(\text{OfflineTestPwd}, \text{sid}, x)$ to $\mathcal{F}_{\text{safPAKE}}$.
 - B. If $\mathcal{F}_{\text{safPAKE}}$ returns pw , retrieve $\text{rw} := \text{rw}[\text{sid}]$, $(k, \{f_i\}) := \text{params}[\text{sid}]$, record $\text{password}[\text{sid}] := \text{pw}$, and run $\text{ProgramDatabase}(\text{sid}, \text{pw}, \text{rw}, k, \{f_i\})$.
 - C. If $\mathcal{S}$ is not marked compromised, $k^* = k$, and $F_{k,1}(\text{password}[\text{sid}])$ is undefined, abort.
 - D. In any other case:
 - I. If undefined, sample $F_{k^*, \text{trans}^*}(x) := f.\text{tag} \| f.\text{ek} \| f.\text{rs} \| f.\text{rw} \leftarrow \{0, 1\}^{4\kappa}$ and abort if there exists another OPRF output with which collides on the tag, ek, rs, or rw.
 - II. Return $(\text{OfflineEval}, \text{sid}, F_{k^*, \text{trans}^*}(x))$ to $\mathcal{A}$ from $\mathcal{F}_{\text{OPRF}}$.
7. Upon receiving $(\text{TestKey}, \text{sid}, k^*)$ from $\mathcal{A}$ sent to $\mathcal{F}_{\text{OPRF}}$:
 - A. If $k = k^*$ where $(\text{DB}, k) := \text{file}[\text{sid}]$, set $\text{P} := \mathcal{S}$.
 - B. Otherwise, set $\text{P} := \perp$.
 - C. Return $(\text{TestKey}, \text{sid}, k^*, \text{P})$ to $\mathcal{A}$ from $\mathcal{F}_{\text{OPRF}}$.
8. Upon receiving $(\text{OfflineTestPwd}, \text{sid}, \text{rw}^*)$ from $\mathcal{A}$ sent to $\mathcal{F}_{\text{aPAKE}}$:
 - A. Add rw^* to $\text{tested}[\text{sid}]$.
 - B. If $\mathcal{S}$ is marked compromised, retrieve $\text{rw} := \text{rw}[\text{sid}]$ ($\text{rw}[\text{sid}]$ is defined when Sim generates $\text{file}[\text{sid}]$, which in turn is no later than $\mathcal{S}$ is compromised):
 - I. If $\text{rw} = \text{rw}^*$, send “correct guess” to $\mathcal{A}$ from $\mathcal{F}_{\text{aPAKE}}$.
 - II. Otherwise, send “wrong guess” to $\mathcal{A}$ from $\mathcal{F}_{\text{aPAKE}}$.

Password Authentication

9. Initialize maps clientSession , serverSession , delayedTests , and delayedSessions .
10. Upon receiving $(\text{ClientSession}, \text{sid}, \text{ssid}, \text{C}, \text{S})$ from $\mathcal{F}_{\text{safPAKE}}$:
 - A. If $\text{clientSession}[\text{ssid}]$ is undefined, set $\text{clientSession}[\text{ssid}] := (\text{C}, \text{S}, \perp, \perp, \perp, \perp, \perp)$; otherwise, ignore the query.

- B. Send $(\text{PRFEval}, \text{sid}, \text{ssid}, \text{C}, \text{S})$ to $\mathcal{A}$ from $\mathcal{F}_{\text{OPRF}}$ and await $(\text{sid}, \text{ssid}, \text{prfx})$ from $\mathcal{A}$. Reject the response unless prfx is new.
 - C. Update $\text{clientSession}[\text{ssid}] := (\text{C}, \text{S}, \perp, \text{prfx}, \perp, \perp, \perp)$ and await $(\text{PRFReceiveComplete}, \text{sid}, \text{ssid}, \text{C}, i)$ from $\mathcal{A}$ sent to $\mathcal{F}_{\text{OPRF}}$.
 - D. If $i = \text{S}$ and $(\text{serverSession}[\text{ssid}] \text{ is undefined or } \text{serverSession}[\text{ssid}] \neq (\text{C}, \text{S}, \cdot, \text{prfx}, \cdot, \cdot))$:
 - I. If $\text{delayedTests}[\text{sid}] = 0$, ignore this message.
 - II. Otherwise, set $\text{delayedTests}[\text{sid}] := \text{delayedTests}[\text{sid}] - 1$.
 - E. If the request wasn't ignored, compute $(\text{state}, \text{req}) := \text{PIR.Request}(0^\kappa)$ and send $(\text{sid}, \text{ssid}, \text{req})$ to $\mathcal{A}$ from C then await $(\text{sid}, \text{ssid}, (\text{resp}, h))$ from $\mathcal{A}$ sent to C .
 - F. Upon receiving $(\text{sid}, \text{ssid}, (\text{resp}, h))$, send $(\text{ClientSession}, \text{sid}, \text{ssid} \parallel \text{prfx} \parallel \text{req} \parallel \text{resp} \parallel h, \text{C}, \text{S})$ to $\mathcal{A}$ from $\mathcal{F}_{\text{aPAKE}}$, set $\text{clientSession}[\text{ssid}] := (\text{C}, \text{S}, i, \text{prfx}, \text{req}, \text{resp}, h)$, and mark $\text{clientSession}[\text{ssid}]$ as "aPAKE active".
11. Upon receiving $(\text{ServerSession}, \text{sid}, \text{ssid}, \text{C}, \text{S})$ from $\mathcal{F}_{\text{safPAKE}}$
- A. If $\text{serverSession}[\text{ssid}]$ is undefined, set $\text{serverSession}[\text{ssid}] := (\text{C}, \text{S}, \perp, \perp, \perp, \perp, \perp)$; otherwise, ignore the query.
 - B. Send $(\text{PRFSendCompletesid}, \text{ssid}, \text{S})$ to $\mathcal{A}$ from $\mathcal{F}_{\text{OPRF}}$, await $(\text{sid}, \text{ssid}, \text{prfx})$ from $\mathcal{A}$, and update $\text{serverSession}[\text{ssid}] := (\text{C}, \text{S}, \perp, \text{prfx}, \perp, \perp, \perp)$.
 - C. If $\text{clientSession}[\text{ssid}] \neq (\text{C}, \text{S}, \cdot, \text{prfx}, \cdot, \cdot, \cdot)$ or if $\text{clientSession}[\text{ssid}]$ is undefined, send $(\text{Interrupt}, \text{sid}, \text{ssid}, \text{S})$ to $\mathcal{F}_{\text{safPAKE}}$, set $\text{delayedTests}[\text{sid}] := \text{delayedTests}[\text{sid}] + 1$, and add ssid to $\text{delayedSessions}[\text{sid}]$.
 - D. Upon receiving $(\text{PRFEval}, \text{sid}, \text{ssid}, \text{S}, x)$ from $\mathcal{A}$ sent to $\mathcal{F}_{\text{OPRF}}$ send $(\text{PRFEval}, \text{sid}, \text{ssid}, \mathcal{A}, \text{S})$ to $\mathcal{A}$ from $\mathcal{F}_{\text{OPRF}}$.
 - I. Await $(\text{PRFReceiveComplete}, \text{sid}, \text{ssid}, \mathcal{A}, i)$ from $\mathcal{A}$ sent to $\mathcal{F}_{\text{OPRF}}$.
 - II. If $i = \text{S}$ and $\text{delayedTests}[\text{sid}] > 0$, set $\text{delayedTests}[\text{sid}] := \text{delayedTests}[\text{sid}] - 1$, choose any sub-session $\text{ssid}' \in \text{delayedSessions}[\text{sid}]$ send $(\text{TestPwd}, \text{sid}, \text{ssid}', \text{S}, x)$ to $\mathcal{F}_{\text{safPAKE}}$ and remove ssid' from $\text{delayedSessions}[\text{sid}]$.
 - III. If $i = (k, 1)$ where $(\text{DB}, k) := \text{file}[\text{sid}]$: If S is not compromised, and password has not been set, abort; otherwise, send $(\text{OfflineTestPwd}, \text{sid}, x)$ to $\mathcal{F}_{\text{safPAKE}}$.
 - a. If $\mathcal{F}_{\text{safPAKE}}$ responds with pw in either case, set $\text{password}[\text{sid}] := \text{pw}$, retrieve $\text{rw} := \text{rw}[\text{sid}]$, $(\text{DB}, k) := \text{file}[\text{sid}]$, $\{f_j\} := \text{params}[\text{sid}]$, and run $\text{ProgramDatabase}(\text{sid}, \text{pw}, \text{rw}, k, \{f_j\})$.
 - b. In any case, if $F_{k,1}(x)$ is undefined, sample $F_{k,1}(x) := f.\text{tag} \parallel f.\text{ek} \parallel f.\text{rs} \parallel f.\text{rw} \leftarrow \{0, 1\}^{4\kappa}$ and abort if there exists another OPRF output with which collides on the tag, ek, rs, or rw.
 - c. Finally, send $(\text{PRFReceiveComplete}, \text{sid}, \text{ssid}, F_{k,1}(x))$ to $\mathcal{A}$.
 - IV. Otherwise, parse i as (k^*, trans^*) and if $F_{k^*, \text{trans}^*}(x)$ is undefined; sample $F_{k^*, \text{trans}^*}(x) := f.\text{tag} \parallel f.\text{ek} \parallel f.\text{rs} \parallel f.\text{rw} \leftarrow \{0, 1\}^{4\kappa}$ and abort if there exists another OPRF output with which collides on the tag, ek, rs, or rw; and send $(\text{PRFReceiveComplete}, \text{sid}, F_{k^*, \text{trans}^*}(x))$ to $\mathcal{A}$.
 - E. Upon receiving $(\text{sid}, \text{ssid}, \text{req})$, compute $\text{resp} := \text{PIR.Respond}(\text{req}, \text{database}[\text{sid}])$ and $h := H(\text{database}[\text{sid}])$, send $(\text{sid}, \text{ssid}, (\text{resp}, h))$ to $\mathcal{A}$ from S , and set $\text{serverSession}[\text{ssid}] := (\text{C}, \text{S}, \text{S}, \text{prfx}, \text{req}, \text{resp}, h)$.

- F. Finally, send $(\text{ServerSession}, \text{sid}, \text{ssid} \parallel \text{prfx} \parallel \text{req} \parallel \text{resp} \parallel h, C, S)$ to $\mathcal{A}$ from $\mathcal{F}_{\text{aPAKE}}$ and mark $\text{serverSession}[\text{ssid}]$ as “aPAKE active”.

Active Session Attacks

12. Upon receiving $(\text{TestPwd}, \text{sid}, \text{ssid} \parallel \text{prfx} \parallel \text{req} \parallel \text{resp} \parallel h, C, \text{rw}^*)$ from $\mathcal{A}$ sent to $\mathcal{F}_{\text{aPAKE}}$, if $(C, S, i, \text{prfx}, \text{req}, \text{resp}, h) := \text{clientSession}[\text{ssid}]$ is marked “aPAKE active”:
 - A. If $i = S$, set $i := (k, 1)$ where $(\text{DB}, k) := \text{file}[\text{sid}]$.
 - B. If $\text{trans}^* = 0$, set $\text{pw}^* := \perp$.
 - C. If there exists an OPRF entry $F_{k^*, \text{trans}^*}(\text{pw}^*) = s \parallel \text{rw}^*$ ($s \in \{0, 1\}^{3\kappa}$), set $\text{pw}^* := x$, otherwise set $\text{pw}^* := \perp$.
 - D. If $i = (k, 1)$, set $\text{DB}^* = \text{DB}$ and if either $\text{password}[\text{sid}]$ is undefined or $\text{pw}^* \neq \text{password}[\text{sid}]$, set $\text{pw}^* := \perp$.
 - E. Otherwise, generate database $\text{DB}^* := \text{DB}(\text{pw}^*, (k^*, \text{trans}^*))$ using $F_{k^*, \text{trans}^*}(\widetilde{\text{pw}})$ for $\widetilde{\text{pw}} \in \Sigma_{\text{pw}^*}$. If any such value is undefined, set $\text{pw}^* := \perp$.
 - F. If either $\text{resp} \neq \text{PIR}.\text{Respond}(\text{DB}^*, \text{req})$ or $h \neq H(\text{DB}^*)$, set $\text{pw}^* := \perp$.
 - G. Send $(\text{TestPwd}, \text{sid}, \text{ssid}, C, \text{pw}^*)$ to $\mathcal{F}_{\text{safPAKE}}$ and if $\mathcal{F}_{\text{safPAKE}}$ responds “wrong guess”, Sim sets $\text{pw}^* := \perp$.
 - H. If $\mathcal{F}_{\text{safPAKE}}$ responds with pw , set $\text{password}[\text{sid}] := \text{pw}$, retrieve $\text{rw} := \text{rw}[\text{sid}]$, $\{f_j\} := \text{params}[\text{sid}]$, and run $\text{ProgramDatabase}(\text{sid}, \text{pw}, \text{rw}, k, \{f_j\})$, mark $\text{clientSession}[\text{ssid}]$ compromised, and send “correct guess” to $\mathcal{A}$.
 - I. Otherwise, send “wrong guess” to $\mathcal{A}$ from $\mathcal{F}_{\text{aPAKE}}$.
 - J. In all cases, mark $\text{clientSession}[\text{ssid}]$ “aPAKE key-ready”.
13. Upon receiving $(\text{TestPwd}, \text{sid}, \text{ssid} \parallel \text{prfx} \parallel \text{req} \parallel \text{resp}, S, \text{rw}^*)$ from $\mathcal{A}$ sent to $\mathcal{F}_{\text{aPAKE}}$, if $(C, S, S, \text{prfx}, \text{req}, \text{resp}, h) := \text{serverSession}[\text{ssid}]$ is marked “aPAKE active”.
 - A. If $\text{rw}[\text{sid}] = \text{rw}^*$, then send “correct guess” to $\mathcal{A}$.
 - B. Otherwise, send “wrong guess” to $\mathcal{A}$.
 - C. In all cases, mark $\text{serverSession}[\text{ssid}]$ “aPAKE key-ready”.
14. Upon receiving $(\text{Impersonate}, \text{sid}, \text{ssid} \parallel \text{prfx} \parallel \text{req} \parallel \text{resp} \parallel h)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{aPAKE}}$, if $(C, S, i, \text{prfx}, \text{req}, \text{resp}, h) := \text{clientSession}[\text{ssid}]$ is marked “aPAKE active”:
 - A. If S is not compromised, ignore this query.
 - B. If $(i = S \text{ or } i = (k, 1))$, $\text{resp} = \text{PIR}.\text{Respond}(\text{req}, \text{DB})$, and $h = H(\text{DB})$, where $(\text{DB}, k) := \text{file}[\text{sid}]$, send $(\text{Impersonate}, \text{sid}, \text{ssid})$ to $\mathcal{F}_{\text{safPAKE}}$ and forward the response (“correct guess” or “wrong guess”) to $\mathcal{A}$.
 - C. If “correct guess” was sent, mark $\text{clientSession}[\text{ssid}]$ compromised.
 - D. If “wrong guess” was sent, send $(\text{TestPwd}, \text{sid}, \text{ssid}, C, \perp)$ to $\mathcal{F}_{\text{safPAKE}}$.
 - E. In both cases, mark $\text{clientSession}[\text{ssid}]$ “aPAKE key-ready”.

Key Generation and Authentication

15. Upon receiving $(\text{NewKey}, \text{sid}, \text{ssid} \parallel \text{prfx}_C \parallel \text{req} \parallel \text{resp} \parallel h, C, k^*)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{aPAKE}}$, and $(C, S, i, \text{prfx}, \text{req}, \text{resp}, h) := \text{clientSession}[\text{ssid}]$ is marked “aPAKE active” or “aPAKE key-ready”:
 - A. If there is a corresponding aPAKE session for the server (i.e., $\text{serverSession}[\text{ssid}] = (C, S, S, \text{prfx}_S, \text{req}', \text{resp}', h')$) and $i \neq S$, $\text{prfx}_C \neq \text{prfx}_S$, $\text{req} \neq \text{req}'$, $\text{resp} \neq \text{resp}'$, or $h \neq h'$, send $(\text{TestPwd}, \text{sid}, \text{ssid}, P, \perp)$ to $\mathcal{F}_{\text{safPAKE}}$.
 - B. Send $(\text{NewKey}, \text{sid}, \text{ssid}, P, k^*)$ to $\mathcal{F}_{\text{safPAKE}}$ and mark $\text{clientSession}[\text{ssid}]$ “aPAKE completed”.
16. Upon receiving $(\text{NewKey}, \text{sid}, \text{ssid} \parallel \text{prfx}_C \parallel \text{req} \parallel \text{resp} \parallel h, S, k^*)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{aPAKE}}$, and $(C, S, i, \text{prfx}, \text{req}, \text{resp}, h) := \text{serverSession}[\text{ssid}]$ is marked “aPAKE active” or “aPAKE key-ready”:
 - A. If there is a corresponding aPAKE session for the client (i.e., $\text{clientSession}[\text{ssid}] = (C, S, S, \text{prfx}_C, \text{req}', \text{resp}', h')$) and $i \neq S$, $\text{req} \neq \text{req}'$, $\text{resp} \neq \text{resp}'$, or $h \neq h'$, send $(\text{TestPwd}, \text{sid}, \text{ssid}, S, \perp)$ to $\mathcal{F}_{\text{safPAKE}}$.
 - B. Send $(\text{NewKey}, \text{sid}, \text{ssid}, P, k^*)$ to $\mathcal{F}_{\text{safPAKE}}$ and mark $\text{serverSession}[\text{ssid}]$ “aPAKE completed”.
17. Upon receiving $(\text{TestAbort}, \text{sid}, \text{ssid} \parallel \text{prfx}_C \parallel \text{req} \parallel \text{resp} \parallel h, S)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{aPAKE}}$, and S has such an active aPAKE session:
 - A. If there is a corresponding aPAKE session for the server (i.e., $\text{serverSession}[\text{ssid}] = (C, S, S, \text{prfx}_S, \text{req}', \text{resp}', h')$) and $i \neq S$, $\text{prfx}_C \neq \text{prfx}_S$, $\text{req} \neq \text{req}'$, $\text{resp} \neq \text{resp}'$, or $h \neq h'$, send $(\text{TestPwd}, \text{sid}, \text{ssid}, P, \perp)$ to $\mathcal{F}_{\text{safPAKE}}$.
 - B. Send $(\text{TestAbort}, \text{sid}, \text{ssid}, C)$ to $\mathcal{F}_{\text{safPAKE}}$, relay the response (“success” or “abort”) to $\mathcal{A}$ from $\mathcal{F}_{\text{aPAKE}}$, and mark C ’s session record “aPAKE completed”.
18. Upon receiving $(\text{TestAbort}, \text{sid}, \text{ssid} \parallel \text{prfx}_S \parallel \text{req} \parallel \text{resp} \parallel h, S)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{aPAKE}}$, and S has such an active aPAKE session:
 - A. If there is a corresponding aPAKE session for the client (i.e., $\text{clientSession}[\text{ssid}] = (C, S, S, \text{prfx}_C, \text{req}', \text{resp}', h')$) and $i \neq S$, $\text{req} \neq \text{req}'$, $\text{resp} \neq \text{resp}'$, or $h \neq h'$, send $(\text{TestPwd}, \text{sid}, \text{ssid}, S, \perp)$ to $\mathcal{F}_{\text{safPAKE}}$.
 - B. Send $(\text{TestAbort}, \text{sid}, \text{ssid}, C)$ to $\mathcal{F}_{\text{safPAKE}}$, relay the response (“success” or “abort”) to $\mathcal{A}$ from $\mathcal{F}_{\text{aPAKE}}$, and mark S ’s session record “aPAKE completed”.

Proof Outline. The proof follows a sequence of indistinguishable hybrids starting in the real world where the parties follow the protocol described in [Section 6](#) and ending with the ideal world where interactions are simulated as per our simulator Sim in [Section 7.1](#) which runs independently of the parties’ passwords.

The first step of our hybrids introduces a dummy party for the client and the server which simply passes messages from the environment to their real counterparts. We then introduce a partial functionality $\mathcal{F}$ which sits between the dummy and real parties. This partial functionality begins by relaying all messages it receives to the simulator, and along the hybrids we build this partial functionality into $\mathcal{F}_{\text{safPAKE}}$. In the beginning, the simulator receives $(\text{StorePwdFile}, \text{sid}, C, \text{pw})$ when the dummy server sends the message to $\mathcal{F}$ and receives $(\text{ClientSession}, \text{sid}, \text{ssid}, S, \text{pw}')$ when the

dummy client sends the message to $\mathcal{F}$ and can run the real world protocol personally giving the keys to each of the parties. After setting up the conceptual functionality party, we give it all necessary interfaces, but provide it a partial **NewKey** interface which we eventually modify into the correct one found in $\mathcal{F}_{\text{safPAKE}}$.

We then modify the client’s code to run independently of the input password. This is accomplished by first replacing the client’s PIR verification step. Instead of verifying the PIR response from the server using the client’s PIR state, we extract the adversary’s input to the aPAKE stage and compare it to the messages sent in the PIR stage (culminating in either a **TestPwd** query or an **Impersonate** query). Now, the client runs independently of its PIR state and by the client privacy property of PIR, we may replace the client’s PIR request and OPRF request with a “dummy” request $(\text{state}, \text{req}) \leftarrow \text{PIR.Request}(0^\kappa)$. After this step, the client’s code runs independently of the client password except for its preliminary query to $\mathcal{F}_{\text{OPRF}}$.

We then modify the server’s code to run independently of the input password unless it receives it from the functionality on a password testing query. To achieve this, we begin by distinguishing sub-sessions where the OPRF prefixes match from ones where they diverge. In the case of divergent sub-sessions, the simulator provides a single **TestPwd** ticket to the environment which can be redeemed to make a password guess. In such divergent sub-sessions, the simulator sends an **Interrupt** query and awaits a **PRFReceiveComplete** query which consumes a ticket. Upon consuming a ticket, the simulator extracts a guess from the OPRF query, sends a **TestPwd** query to the functionality, and aligns the aPAKE session accordingly. In the next set of hybrids, we replace the server’s storage with a uniformly generated database independent of the server’s password. We then align these databases whenever a ticket is redeemed, resulting in a correct password guess against the server, by programming the OPRF outputs of all passwords $\widetilde{pw} \in \Sigma_{pw}$ to match those used in the generation of the database.

Finally, we modify the partial functionality to align with $\mathcal{F}_{\text{safPAKE}}$. This is done, first, by replacing the intermediate **NewKey** interface in the partial simulator with the one found in $\mathcal{F}_{\text{safPAKE}}$. Once this is accomplished, we modify the functionality to no longer provide the simulator with the full **ClientSession** and **ServerSession** messages. As the simulator still relies on these messages, we make the final modifications to the simulator. First, we modify the server’s execution to generate the dummy database upon receiving a successful **StealPwdFile** query or a **ServerSession** query. We then modify the client’s code to no longer make its first OPRF query and for the simulator to instead send a **PRFEval** message to the environment without prompting. To finish the hybrids, we modify the partial functionality to no longer prompt the simulator by sending **StorePwdFile**. $\square$

We present a complete proof of [Theorem 3](#) in [Appendix A](#). We additionally note, as a conjecture, that if the client leakage function $\ell_C(pw, pw') = \text{“correct password”}$ is replaced with the server’s leakage function $\ell_S(pw, pw') = pw$, then we may achieve a more efficient protocol which no longer requires the client to verify the server’s PIR response. This also means that the PIR scheme would not need to have deterministic messages; however, this leakage function is no longer *minimal* in that it does not match what the honest parties learn at the end of the protocol.

8 Concrete Instantiations & Efficiency

We now discuss concrete instantiations for the components of our protocol (PIR scheme and aPAKE), and the resulting performance.

Choice of PIR scheme. We consider two PIR schemes:

- **Trivial PIR:** The server sends the entire database to the client. Communication equals the size of the database.
- **FHE-PIR:** The client sends an FHE encryption of its query (tag) q to the server. The server homomorphically evaluates $P(q)$, where P is a polynomial obtained by interpolating $P(x) = DB[x]$. Communication is the size of one FHE-encrypted database element in each direction. Computation for the server is $O(|DB|)$ FHE operations.

Other PIR protocols between these extremes are possible. For example, it is possible to exponentially reduce the multiplicative depth of the FHE circuit by having the client send FHE encryptions of $q, q^2, q^4, q^8, \dots$ instead of just an FHE encryption of q . This variant trades a $\log |DB|$ factor increase in communication for a significant reduction in FHE costs.

In our protocol, database items are symmetric-key encrypted ciphertexts, where the plaintext contains a PRF key and a small integer. Pessimistically, we consider each ciphertext to be 3κ bits, and we assume an additive overhead of $O(\kappa)$ to further encrypt the database item under FHE.

Round complexity of safPAKE. Our OPRF protocol consists of two sequential rounds, with the client speaking first. Each OPRF message is a single group element. Afterwards, the rest of our protocol contains several sequential bottlenecks that affect the total round complexity:

- The client must obtain its OPRF output before making its PIR query. However, when using trivial-PIR, the client makes no query, so the server can just send its trivial-PIR message together with its final OPRF message.
- The client must obtain its PIR output before sending its first aPAKE message. However, if the server is first to speak in the aPAKE protocol, it may send the first aPAKE message along with its PIR response. Hence, our protocol favors aPAKE in which the server speaks first.

Overall, our protocol requires $2 + 2P + A - S$ rounds, where $P = 1$ for nontrivial PIR ($P = 0$ otherwise), the aPAKE has A sequential rounds, and $S = 1$ if the server speaks first in the aPAKE ($S = 0$ otherwise).

Among aPAKE protocols, the one of Hwang et al. [HJK⁺18] instantiated with the PAKE protocol of McQuoid et al. [MRR20] is best suited to our application. It requires only one message from each party, and the server may be first to speak. Its communication cost is 7κ bits more than bare Diffie-Hellman, and it requires only one more exponentiation for the client than bare Diffie-Hellman.

Summary of safPAKE costs. In Table 1 we show the concrete costs of our protocol when instantiated according to our recommendations here vs the naïve safPAKE described previously instantiated with OPAQUE [JKX18b] and reusing the OPRF stage. The reader should remember that the resultant protocol does not actually securely UC-realize the $\mathcal{F}_{\text{safPAKE}}$ functionality as there are no guarantees that the server uses the correct inputs. Importantly, when using a nontrivial PIR, the communication cost of our protocol is independent of the number of allowed password variants. When using a trivial PIR, communication is linear in the number of password variants, but the server’s computation is constant.

Much of the computational cost comes from the client re-computing the PIR database. This involves $O(|DB|)$ PRF evaluations, each of which requires an exponentiation. In Table 1 we list our costs in terms of exponentiations E , hash-to-group operations H , and FHE multiplications F

as these are our computational bottlenecks. Here, by n we mean the size of the typo-set Σ_{pw} — recall that this should be constant across all passwords.

	Naïve + [JKX18a]	Ours + Trivial PIR	Ours + FHE PIR
C Cost	$3n + 2E, 1H$	$(n + 4)E, (n + 3)H$	$(n + 4)E, (n + 3)H, O(n)F$
S Cost	$3n + 1E$	$3E, 2H$	$3E, 2H, nF$
Rounds	3	3	5
Communication	$(2n + 2)G + (5n + 2)\kappa$	$(3n + 9)\kappa + 4G$	$O(\kappa) + 4G + 9\kappa$
S Storage	$(5n + 1)\kappa + 2nG$	$(3n + 1)\kappa$	$O(\kappa)n + (3n + 1)\kappa$

Table 1: Cost evaluation of our safPAKE protocol. The “naïve” protocol is the insecure protocol described in [Section 1](#).

Typo Policies: Our protocol is suitable for typo policies with a small or moderate number of allowable typos. Here we propose several such policies:

- **Facebook’s policy:** Facebook allows the client to enter either a case-reversed password, a repeated first/last character password, or a case-reversed first letter password so, including the correct password, $\Sigma_{pw} = 5$ total password variants will be accepted⁸.
- **Keyboard-adjacent character substitutions:** Suppose we allow at most k characters of the length- $\ell \geq k$ password to be substituted for characters that are adjacent on the keyboard, then $\Sigma_{pw} = \sum_{i \leq k} \binom{\ell}{i} 7^i$ total password variants will be accepted.
- **Consecutive characters transposed:** Suppose we allow at most k pairs of consecutive characters in the length- $\ell \geq k$ password to be transposed, then $\Sigma_{pw} = \sum_{i \leq k} \binom{\ell-1}{i}$ total password variants will be accepted.
- **Erroneous repeated or dropped characters:** Suppose we allow at most k characters in the length- $\ell \geq k$ password to be repeated or dropped, then $\Sigma_{pw} = \sum_{i \leq k} \binom{\ell}{i} 2^i$ total password variants will be accepted.

If we consider the cost of implementing Facebook’s simple password-typo policy using our scheme, instantiated with the trivial PIR and a 256-bit elliptic curve group $\kappa = 128$, authentication would require 3 rounds, $((3 \cdot 5 + 9) \cdot 128 + 4 \cdot 256)/8 = 512$ bytes of communication, 12 exponentiations, 10 hash-to-curve operations, and 256 bytes of server storage.

9 Conclusion

Following in a line of study for fuzzy symmetric PAKE [DHP⁺18] and fuzzy asymmetric PAKE [PC20, EHOR20], we present a functionality for fuzzy *strong* asymmetric PAKE which affords the same strong guarantees of the saPAKE functionality from the literature [JKX18a] while allowing for arbitrary measures of similarity. We additionally provide a construction in the spirit of the aPAKE to saPAKE compiler due to Jarecki, Krawczyk, and Xu [JKX18a] and tPAKE due to Pongmorakot and Chatterjee [PC20]. Our protocol realizing the $\mathcal{F}_{\text{safPAKE}}$ functionality for a

⁸This count doesn’t hold for all passwords, but will hold for the majority of passwords. e.g., the empty password will have $\Sigma_{pw} = 1$.

common subclass of the similarity predicates considered by Erwig et al. [EHOR20] and can be built from a variant of the 2-hash-Diffie-Hellman OPRF herein called 3-hash-Diffie-Hellman, keyword private information retrieval, and existing aPAKE protocols. Additionally, we provide an incidental protocol for strong asymmetric normalization which we believe to be an independent interest for future study.

Our construction can achieve constant communication size in the size of the server’s database, however has client computation equal to constructing the server’s PIR database. For simple predicates such as first word capitalization or first-second letter transposition, this is reasonable, but for biometric applications may be too expensive for deployment. We believe that reducing the computational burden for the client is possible and may pave the path to strong asymmetric biometric authentication. We leave this as an interesting line for future work.

References

- [ABB⁺20] Michel Abdalla, Manuel Barbosa, Tatiana Bradley, Stanislaw Jarecki, Jonathan Katz, and Jiayu Xu. Universally composable relaxed password authenticated key exchange. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part I*, volume 12170 of *LNCS*, pages 278–307. Springer, Cham, August 2020.
- [ABK⁺21] Michel Abdalla, Manuel Barbosa, Jonathan Katz, Julian Loss, and Jiayu Xu. Algebraic adversaries in the universal composability framework. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021*, volume 13092 of *LNCS*, pages 311–341. Springer, 2021.
- [AHH21] Michel Abdalla, Björn Haase, and Julia Hesse. Security analysis of CPace. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part IV*, volume 13093 of *LNCS*, pages 711–741. Springer, Cham, December 2021.
- [AP05] Michel Abdalla and David Pointcheval. Simple password-based encrypted key exchange protocols. In Alfred Menezes, editor, *CT-RSA 2005*, volume 3376 of *LNCS*, pages 191–208. Springer, Berlin, Heidelberg, February 2005.
- [BFH⁺23] Jonathan Bootle, Sebastian H. Faller, Julia Hesse, Kristina Hostáková, and Johannes Ottenhues. Generalized fuzzy password-authenticated key exchange from error correcting codes. In Jian Guo and Ron Steinfeld, editors, *ASIACRYPT 2023, Part VIII*, volume 14445 of *LNCS*, pages 110–142. Springer, Singapore, December 2023.
- [BJX19] Tatiana Bradley, Stanislaw Jarecki, and Jiayu Xu. Strong asymmetric PAKE based on trapdoor CKEM. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part III*, volume 11694 of *LNCS*, pages 798–825. Springer, Cham, August 2019.
- [BM93] Steven M. Bellovin and Michael Merritt. Augmented encrypted key exchange: A password-based protocol secure against dictionary attacks and password file compromise. In Dorothy E. Denning, Raymond Pyle, Ravi Ganesan, Ravi S. Sandhu, and Victoria Ashby, editors, *ACM CCS 93*, pages 244–250. ACM Press, November 1993.
- [BMP00] Victor Boyko, Philip D. MacKenzie, and Sarvar Patel. Provably secure password-authenticated key exchange using Diffie-Hellman. In Bart Preneel, editor, *EUROCRYPT 2000*, volume 1807 of *LNCS*, pages 156–171. Springer, Berlin, Heidelberg, May 2000.

- [BNPS03] Mihir Bellare, Chanathip Namprempre, David Pointcheval, and Michael Semanko. The one-more-RSA-inversion problems and the security of Chaum’s blind signature scheme. *Journal of Cryptology*, 16(3):185–215, June 2003.
- [BPR00] Mihir Bellare, David Pointcheval, and Phillip Rogaway. Authenticated key exchange secure against dictionary attacks. In Bart Preneel, editor, *EUROCRYPT 2000*, volume 1807 of *LNCS*, pages 139–155. Springer, Berlin, Heidelberg, May 2000.
- [CAA⁺16] Rahul Chatterjee, Anish Athayle, Devdatta Akhawe, Ari Juels, and Thomas Ristenpart. pASSWORD tYPOS and how to correct them securely. In *2016 IEEE Symposium on Security and Privacy*, pages 799–818. IEEE Computer Society Press, May 2016.
- [CHK⁺05] Ran Canetti, Shai Halevi, Jonathan Katz, Yehuda Lindell, and Philip D. MacKenzie. Universally composable password-based key exchange. In Ronald Cramer, editor, *EUROCRYPT 2005*, volume 3494 of *LNCS*, pages 404–421. Springer, Berlin, Heidelberg, May 2005.
- [DHP⁺18] Pierre-Alain Dupont, Julia Hesse, David Pointcheval, Leonid Reyzin, and Sophia Yakoubov. Fuzzy password-authenticated key exchange. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part III*, volume 10822 of *LNCS*, pages 393–424. Springer, Cham, April / May 2018.
- [EHOR20] Andreas Erwig, Julia Hesse, Maximilian Ortl, and Siavash Riahi. Fuzzy asymmetric password-authenticated key exchange. In Shiho Moriai and Huaxiong Wang, editors, *ASIACRYPT 2020, Part II*, volume 12492 of *LNCS*, pages 761–784. Springer, Cham, December 2020.
- [GJK21] Yanqi Gu, Stanislaw Jarecki, and Hugo Krawczyk. KHAPE: Asymmetric PAKE from key-hiding key exchange. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part IV*, volume 12828 of *LNCS*, pages 701–730, Virtual Event, August 2021. Springer, Cham.
- [GMR06] Craig Gentry, Philip MacKenzie, and Zulfikar Ramzan. A method for making password-based key exchange resilient to server compromise. In Cynthia Dwork, editor, *CRYPTO 2006*, volume 4117 of *LNCS*, pages 142–159. Springer, Berlin, Heidelberg, August 2006.
- [Her22] Amir Herzberg. Public-key infrastructure (PKI), 2022. https://uconn-my.sharepoint.com/:p:/g/personal/amir_herzberg_uconn_edu1/EX99yfXIWrtNqPo84-auJA4B-ZMLr4JHWZ-Zg83hvIv40w?e=fh0dGk.
- [Hes20] Julia Hesse. Separating symmetric and asymmetric password-authenticated key exchange. In Clemente Galdi and Vladimir Kolesnikov, editors, *SCN 20*, volume 12238 of *LNCS*, pages 579–599. Springer, Cham, September 2020.
- [HJK⁺18] Jung Yeon Hwang, Stanislaw Jarecki, Taekyoung Kwon, Joohee Lee, Ji Sun Shin, and Jiayu Xu. Round-reduced modular construction of asymmetric password-authenticated key exchange. In Dario Catalano and Roberto De Prisco, editors, *SCN 18*, volume 11035 of *LNCS*, pages 485–504. Springer, Cham, September 2018.
- [JKK14] Stanislaw Jarecki, Aggelos Kiayias, and Hugo Krawczyk. Round-optimal password-protected secret sharing and T-PAKE in the password-only model. In Palash Sarkar

- and Tetsu Iwata, editors, *ASIACRYPT 2014, Part II*, volume 8874 of *LNCS*, pages 233–253. Springer, Berlin, Heidelberg, December 2014.
- [JKKX17] Stanislaw Jarecki, Aggelos Kiayias, Hugo Krawczyk, and Jiayu Xu. TOPPSS: Cost-minimal password-protected secret sharing based on threshold OPRF. In Dieter Gollmann, Atsuko Miyaji, and Hiroaki Kikuchi, editors, *ACNS 2017*, volume 10355 of *LNCS*, pages 39–58. Springer, Cham, July 2017.
 - [JKX18a] Stanislaw Jarecki, Hugo Krawczyk, and Jiayu Xu. OPAQUE: An asymmetric PAKE protocol secure against pre-computation attacks. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part III*, volume 10822 of *LNCS*, pages 456–486. Springer, Cham, April / May 2018.
 - [JKX18b] Stanislaw Jarecki, Hugo Krawczyk, and Jiayu Xu. OPAQUE: An asymmetric PAKE protocol secure against pre-computation attacks. Cryptology ePrint Archive, Report 2018/163, 2018.
 - [KOY01] Jonathan Katz, Rafail Ostrovsky, and Moti Yung. Efficient password-authenticated key exchange using human-memorable passwords. In Birgit Pfitzmann, editor, *EUROCRYPT 2001*, volume 2045 of *LNCS*, pages 475–494. Springer, Berlin, Heidelberg, May 2001.
 - [KP01] Charlie Kaufman and Radia J. Perlman. PDM: A new strong password-based protocol. In Dan S. Wallach, editor, *USENIX Security 2001*. USENIX Association, August 2001.
 - [KV11] Jonathan Katz and Vinod Vaikuntanathan. Round-optimal password-based authenticated key exchange. In Yuval Ishai, editor, *TCC 2011*, volume 6597 of *LNCS*, pages 293–310. Springer, Berlin, Heidelberg, March 2011.
 - [MRR20] Ian McQuoid, Mike Rosulek, and Lawrence Roy. Minimal symmetric PAKE and 1-out-of-N OT from programmable-once public functions. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *ACM CCS 2020*, pages 425–442. ACM Press, November 2020.
 - [PC20] Thitikorn Pongmorrakot and Rahul Chatterjee. tpake: Typo-tolerant password-authenticated key exchange. In Lejla Batina, Stjepan Picek, and Mainack Mondal, editors, *SPACE 2020*, volume 12586 of *LNCS*, pages 3–24. Springer, 2020.
 - [Pro11] Emil Protalinski. Facebook passwords are not case sensitive, 2011. <https://www.zdnet.com/article/facebook-passwords-are-not-case-sensitive-update/>.
 - [SGJK22] Bruno Freitas Dos Santos, Yanqi Gu, Stanislaw Jarecki, and Hugo Krawczyk. Asymmetric PAKE with low computation and communication. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part II*, volume 13276 of *LNCS*, pages 127–156. Springer, Cham, May / June 2022.
 - [Sho20] Victor Shoup. Security analysis of spake2+. In Rafael Pass and Krzysztof Pietrzak, editors, *Theory of Cryptography*, pages 31–60, Cham, 2020. Springer International Publishing.

A Proof of Theorem 3

In this section we provide the full proof of Theorem 3. We find the following lemma useful which, in the context of our protocol, states that a given entry $f.\text{tag} \mapsto \mathcal{E}.\text{Enc}(f.\text{ek}; (k, d); f.\text{rs})$ can only exist in at most one valid database.

Lemma 4. *For all similarity predicates σ which have input independent size (with $n > 0$) and predictable hints, the function*

$$\iota_\sigma(x) \mapsto \{(\tilde{x}, \sigma(x, \tilde{x})) \mid \tilde{x} \in \Sigma_x\}$$

produces pairwise disjoint sets.

Proof. Given $x, x' \in \tilde{D}$ such that $I = (\iota_\sigma(x) \cap \iota_\sigma(x')) \neq \emptyset$, there exists some $\tilde{x}$ with $\sigma(x, \tilde{x}) = \sigma(x', \tilde{x}) \neq \perp$. By correctness of σ , we have that $x = \rho(\tilde{x}, \sigma(x, \tilde{x})) = \rho(\tilde{x}, \sigma(x', \tilde{x})) = x'$. $\square$

WLOG, we consider the case where both honest parties C and S are activated in each session. This is because the simulation of a corrupt party is the same as for a MitM which replaces messages in the same pattern as the corrupt party. Similarly, following convention [JKX18a] we assume that whenever $\mathcal{Z}$ compromises either $\mathcal{F}_{\text{aPAKE}}$ or S it also compromises the other.

Proof. Hybrid 0: Real World

In the real world, the protocol depicted in Section 6 is run between two parties S and C who both have access to ideal functionalities $\mathcal{F}_{\text{aPAKE}}$, $\mathcal{F}_{\text{RO}}$, and $\mathcal{F}_{\text{OPRF}}$.

Hybrid 1: Simulating Functionalities

Changes to $\mathcal{F}$: We introduce the partial functionality $\mathcal{F}$ and the dummy parties for the client and server. $\mathcal{F}$ simply relays messages between the simulated and dummy parties.

Changes to Sim: We introduce Sim which encompasses the ideal adversary $\mathcal{A}$, the real parties C and S, and the hybrid functionalities $\mathcal{F}_{\text{OPRF}}$, $\mathcal{F}_{\text{RO}}$, and $\mathcal{F}_{\text{aPAKE}}$. Sim faithfully simulates the hybrid functionalities and acts as the honest Client and Server in the real world protocol on inputs (StorePwdFile, sid, C, pw) and (ClientSession, sid, ssid, S, pw') from $\mathcal{F}$. Additionally, whenever Sim catalogs an input-output pair (x, y) for $\mathcal{F}_{\text{OPRF}}$ or $\mathcal{F}_{\text{RO}}$, it simultaneously records $F_{k, \text{trans}}(x) := y$ or $H(x) := y$ respectively — interpreting $i = S$ as $i = (k, 1)$ for server OPRF key k .

Indistinguishability: These changes are conceptual and do not affect the input-output behavior or messages of the parties, so Hybrid 1 is perfectly indistinguishable from Hybrid 0.

$$\text{Dist}_{\mathcal{Z}}^{0,1} = 0$$

Hybrid 2: Keeping Session State

Changes to $\mathcal{F}$: $\mathcal{F}$ now records internal state for queries from the honest parties while still passing messages to the simulator. Upon receiving a query in {StorePwdFile, ClientSession, ServerSession}, $\mathcal{F}$ records internal state consistent with $\mathcal{F}_{\text{saPAKE}}$ depicted in Figure 3 and Figure 4.

Indistinguishability: $\mathcal{F}$ still relays the messages to Sim, so these changes are completely conceptual and Hybrid 2 is perfectly indistinguishable from Hybrid 1.

$$\text{Dist}_{\mathcal{Z}}^{1,2} = 0$$

Hybrid 3: Add Adversary's Interfaces

Changes to $\mathcal{F}$: We now add the code for all queries in $\{\text{StealPwdFile}, \text{OfflineTestPwd}, \text{Interrupt}, \text{TestPwd}, \text{Impersonate}, \text{TestAbort}\}$ to $\mathcal{F}$ exactly as they are in $\mathcal{F}_{\text{safPAKE}}$.

Indistinguishability: Note that all these queries can only be sent by the ideal adversary (in our case, this is Sim) and since Sim does not make any of these queries, there is no change from $\mathcal{Z}$'s view. Therefore, **Hybrid 3** is perfectly indistinguishable from **Hybrid 2**.

$$\text{Dist}_{\mathcal{Z}}^{2,3} = 0$$

Hybrid 4: Adding Partial NewKey

Changes to $\mathcal{F}$: We change $\mathcal{F}$ to have a modified NewKey interface which differs from that found in $\mathcal{F}_{\text{safPAKE}}$:

Upon receiving $(\text{NewKey}, \text{sid}, \text{ssid}, \text{P}, k^*)$ from $\mathcal{A}^*$ where $\|k^*\| = \kappa$:

1. If $\text{sessionStatus}[(\text{sid}, \text{ssid}, \text{P}, \text{P}')] is defined, but is not completed:

 1. If the record is compromised, or if $(\text{P} = \text{S}, \text{ the record is interrupted, and flag = compromised})$, set $k = k^*$.
 2. Else, if the record is interrupted, sample $k \leftarrow \{0, 1\}^\kappa$.
 3. Else, set $k = k^*$.
 4. In all cases, set $\text{sessionStatus}[(\text{sid}, \text{ssid}, \text{P}, \text{P}')] := \text{completed}$ and send $(\text{sid}, \text{ssid}, k)$ to P.$

Note that, now, the difference between the partial functionality $\mathcal{F}$ and $\mathcal{F}_{\text{safPAKE}}$ are as follows:

- $\mathcal{F}$ relays all messages received to Sim.
- $\mathcal{F}$'s NewKey interface only handles interrupted and compromised sessions correctly.

Changes to Sim: Now Sim uses $\mathcal{F}$'s NewKey interface to communicate keys to the respective parties instead of passing $(\text{sid}, \text{ssid}, k)$ to P.

Indistinguishability: As Sim never makes a query causing flag to be set compromised or any session to be marked interrupted or compromised, **Hybrid 4** is perfectly indistinguishable from **Hybrid 3**.

$$\text{Dist}_{\mathcal{Z}}^{3,4} = 0$$

Hybrid 5: Removing Collisions

Changes to Sim: Upon simulation of $\mathcal{F}_{\text{OPRF}}$ and $\mathcal{F}_{\text{RO}}$, if there exist $f.\text{tag}||f.\text{ek}||f.\text{rs}||f.\text{rw} := F_{k,\text{trans}}(x)$, $f'.\text{tag}||f'.\text{ek}||f'.\text{rs}||f'.\text{rw} := F_{k',\text{trans}}(x')$, for distinct queries, such that $[f.\text{tag} = f'.\text{tag} \vee f.\text{ek} = f'.\text{ek} \vee f.\text{rs} = f'.\text{rs} \vee f.\text{rw} = f'.\text{rw}]$, or if there exists $h := H(x), h' := H(x')$, for distinct queries, such that $h = h'$, Sim aborts.

Indistinguishability: As each fresh substring is uniformly sampled from $\{0, 1\}^\kappa$, collisions happen with negligible probability by a birthday bound.

$$\text{Dist}_{\mathcal{Z}}^{4,5} \leq \frac{4 \cdot q_{\text{OPRF}}^2 + q_{\text{RO}}^2}{2^\kappa}$$

Hybrid 6: Starting the Client's aPAKE Session Early

Changes to Sim: As soon as C is "aPAKE ready" i.e.,

- (Prefix, sid, ssid, $prfx_C$) was sent to C from $\mathcal{F}_{\text{OPRF}}$
- $\mathcal{Z}$ sent (PRFReceiveComplete, sid, ssid, C, i) to $\mathcal{F}_{\text{OPRF}}$
- C sent PIR request req to S
- $\mathcal{Z}$ provided server PIR response $\text{resp}_{\mathcal{Z}}$ and digest $h_{\mathcal{Z}}$ to C

Sim immediately sends (ClientSession, sid, ssid|| $prfx_C$ ||req|| $\text{resp}_{\mathcal{Z}}$ || $h_{\mathcal{Z}}$, C, S) to $\mathcal{Z}$ from $\mathcal{F}_{\text{aPAKE}}$ (we will refer to any honest party who sends such a query as “aPAKE active” for as long as the aPAKE session is fresh). Then on receiving TestPwd, Impersonate, NewKey, or TestAbort from $\mathcal{Z}$ to $\mathcal{F}_{\text{aPAKE}}$, Sim runs the C’s code generating rw' and simulates $\mathcal{F}_{\text{aPAKE}}$ relative to client input rw' .

Indistinguishability: As C will start an aPAKE session independent of whether any check it performs fails, **Hybrid 6** is simply a reordering of private computation for C and is perfectly indistinguishable from **Hybrid 5**.

$$\text{Dist}_{\mathcal{Z}}^{5,6} = 0$$

Hybrid 7: Simulating Client PIR Verification (Dictionary Attacks)

In the next two hybrids, we modify Sim so that instead of verifying the server’s PIR response relative to the client password pw' as in the real protocol, Sim instead extracts a password pw^* from the aPAKE stage and verifies the server’s PIR response relative to pw^* . In this hybrid, we modify Sim to handle dictionary attacks against the client session.

Changes to Sim: If C is “aPAKE active” i.e.,

- (Prefix, sid, ssid, $prfx_C$) was sent to C from $\mathcal{F}_{\text{OPRF}}$
- Sim sent (PRFReceiveComplete, sid, ssid, y) to C in response to (PRFReceiveComplete, sid, ssid, C, i) from $\mathcal{Z}$ to $\mathcal{F}_{\text{OPRF}}$
- C sent PIR request req to S
- $\mathcal{Z}$ provided server PIR response $\text{resp}_{\mathcal{Z}}$ and digest $h_{\mathcal{Z}}$ to C
- Sim sent (ClientSession, sid, ssid|| $prfx_C$ ||req|| $\text{resp}_{\mathcal{Z}}$ || $h_{\mathcal{Z}}$, C, S) to $\mathcal{Z}$ from $\mathcal{F}_{\text{aPAKE}}$
- The aPAKE session is fresh

and $\mathcal{Z}$ sends (TestPwd, sid, ssid|| $prfx_C$ ||req|| $\text{resp}_{\mathcal{Z}}$ || $h_{\mathcal{Z}}$, C, rw^*) to $\mathcal{F}_{\text{aPAKE}}$: if $i = S$, Sim sets $(k^*, \text{trans}^*) := (k, 1)$ for server-stored OPRF key k ; otherwise, Sim interprets $(k^*, \text{trans}^*) := i$.

1. If $\text{trans}^* = 0$, Sim sets $pw^* := \perp$.
2. If there exists an OPRF entry $F_{k^*, \text{trans}^*}(pw^*) = s||rw^*$ ($s \in \{0, 1\}^{3\kappa}$), Sim sets $pw^* := x$, otherwise set $pw^* := \perp$.
3. If $\text{resp}_{\mathcal{Z}} = \text{resp}$, $h_{\mathcal{Z}} = h$, and $i = (k, 1)$ for server key k , Sim sets $\text{DB}^* = \text{DB}$ and if $pw^* \neq pw$ for server password pw , sets $pw^* := \perp$.
4. Otherwise, Sim generates database $\text{DB}^* := \text{DB}(pw^*, (k^*, \text{trans}^*))$ using $F_{k^*, \text{trans}^*}(\widetilde{pw})$ for $\widetilde{pw} \in \Sigma_{pw^*}$. If any such value is undefined, Sim sets $pw^* := \perp$.
5. Sim verifies that $\text{resp}_{\mathcal{Z}} = \text{PIR.Respond}(\text{DB}^*, \text{req})$ and $h_{\mathcal{Z}} = H(\text{DB}^*)$. If either check fails, Sim sets $pw^* := \perp$.
6. Sim sends (TestPwd, sid, ssid, C, pw^*) to $\mathcal{F}_{\text{safPAKE}}$ and if $\mathcal{F}_{\text{safPAKE}}$ responds “wrong guess”, Sim sets $pw^* := \perp$.

If $pw^* \neq \perp$, Sim sets $rw' := rw^*$ and sets $rw' := \perp$ otherwise. Finally, Sim proceeds the aPAKE simulation relative to client input rw' . Note that although Sim's simulation of C still queries the OPRF on pw' , the value is discarded when Sim sets rw' .

Indistinguishability: We recall how C derives aPAKE input rw' in **Hybrid 6**:

- If C fails to compute an OPRF key k' from the PIR interaction, C will uniformly sample rw' .
- If C computes OPRF key k' from the PIR interaction and $(k^*, \text{trans}^*) \neq (k', 1)$, then as $F_{k^*, \text{trans}^*}(pw') \neq F_{k', 1}(pw')$, C will uniformly sample rw' .
- When C reconstructs password $\widetilde{pw}$ along with database $DB' := \text{DB}(\widetilde{pw}, (k', 1))$ and either $\text{resp}_{\mathcal{Z}} \neq \text{PIR.Respond}(DB', \text{req})$ or $h_{\mathcal{Z}} \neq H(DB')$, C will uniformly sample rw' .
- In any other case, C will compute rw' by offline query to $\mathcal{F}_{\text{OPRF}}$ on $\widetilde{pw}$ for $i = (k', 1)$.

In the case that Sim sets $rw' := rw^*$, the honest client will have derived the same aPAKE input. In this case, **Hybrid 7** is identically distributed to **Hybrid 6**. It remains to analyze the case that Sim sets $rw' := \perp$.

- If $\text{trans}^* = 0$, $(k^*, \text{trans}^*) \neq (k', 1)$ and C will uniformly sample rw' .
- If there exists no preimage $F_{k^*, \text{trans}^*}(pw^*) = s || rw^*$, then if C doesn't uniformly sample rw' , it will compute a *new* rw' which is independent of $\mathcal{Z}$'s view.
- If $\mathcal{Z}$ has not learned OPRF output $F_{k^*, \text{trans}^*}(\widetilde{pw})$ for some $\widetilde{pw} \in \Sigma_{pw^*}$, and $\text{resp}_{\mathcal{Z}} \neq \text{resp}$ or $h_{\mathcal{Z}} \neq h$, then when C generates $DB^* = \text{DB}(pw^*, (k^*, \text{trans}^*))$, OPRF output $f_{\widetilde{pw}.\text{tag}} || f_{\widetilde{pw}.\text{ek}} || f_{\widetilde{pw}.\text{rs}} || f_{\widetilde{pw}.\text{rw}} := F_{k^*, \text{trans}^*}(\widetilde{pw})$ will be uniform from $\mathcal{Z}$'s view. As honestly generated databases have pairwise-disjoint entries by **Lemma 4** and, by the AE security of $\mathcal{E}$ (**Definition 1**), the ciphertext $\mathcal{E}.\text{Enc}(f_{\widetilde{pw}.\text{ek}}; (k^*, \sigma(pw^*, \widetilde{pw})); f_{\widetilde{pw}.\text{rs}})$ is unpredictable, $h_{\mathcal{Z}} = H(DB^*)$ with negligible probability.⁹
- Similarly, as generation of the PIR database is deterministic, if C computes $rw' = rw^*$, C also computed DB^* and verified $\text{resp}_{\mathcal{Z}} = \text{PIR.Respond}(DB^*, \text{req})$ and $h_{\mathcal{Z}} = H(DB^*)$. If $\mathcal{Z}$ made no such random oracle query and $h_{\mathcal{Z}} \neq h$, then as $H(DB^*)$ is sampled uniformly, $h_{\mathcal{Z}}$ was chosen correctly with negligible probability.
- If, instead, $\text{resp}_{\mathcal{Z}} = \text{resp}$, $h_{\mathcal{Z}} = h$, and $i = (k, 1)$, then **Lemma 4** guarantees that if C doesn't sample rw' on their own, $rw' = rw$ for server aPAKE storage rw . Therefore, if $pw^* \neq pw$, $rw^* \neq rw'$ with overwhelming probability.
- Finally, in the case that $pw' \notin \Sigma_{pw^*}$, **Lemma 4** guarantees that the databases are disparate, and therefore the two digests diverge: $h' \neq h_{\mathcal{Z}}$.

This triggers the “Else, if the record is interrupted, sample $k \leftarrow \{0, 1\}^{\kappa}$ ” clause in $\mathcal{F}$. In **Hybrid 6**, the outputs of parties with non-matching inputs to the aPAKE state were uniformly distributed through the simulation of $\mathcal{F}_{\text{aPAKE}}$, so there is no distinguishable difference in C's output.

$$\mathbf{Dist}_{\mathcal{Z}}^{6,7} = \frac{2}{2^{\kappa}} + \mathbf{Adv}_{\mathcal{Z}}^{\text{AE}}$$

⁹The only ciphertexts $\mathcal{Z}$ sees that it hasn't generated are those from the honest server. **Lemma 4** guarantees that no ciphertexts overlap between two databases, so $\mathcal{Z}$ hasn't seen the ciphertext corresponding to $pw^*, \widetilde{pw}, (k^*, 1)$ and the reduction immediately follows.

Hybrid 8: Simulating Client PIR Verification (Impersonation Attacks)

In this hybrid, we modify Sim to handle impersonation attacks against the client.

Changes to Sim: Assuming that C is “aPAKE active” i.e.,

- (Prefix, sid, ssid, $prfx_C$) was sent to C from $\mathcal{F}_{\text{OPRF}}$
- Sim sent (PRFReceiveComplete, sid, ssid, y) to C in response to (PRFReceiveComplete, sid, ssid, C, i) from $\mathcal{Z}$ to $\mathcal{F}_{\text{OPRF}}$
- C sent PIR request req to S
- $\mathcal{Z}$ provided server PIR response resp_Z to C
- Sim sent (ClientSession, sid, ssid || $prfx_C$ || req || resp_Z || h_Z , C, S) to $\mathcal{Z}$ from $\mathcal{F}_{\text{aPAKE}}$
- The aPAKE session is fresh

and $\mathcal{Z}$ previously sent (StealPwdFile, sid) to $\mathcal{F}_{\text{aPAKE}}$ and (StealPwdFile, sid) to S with S responding (DB, k), if

- $i = (k, 1)$ or $i = S$
- $\text{resp}_Z = \text{PIR.Request}(\text{DB}, \text{req})$
- $h_Z = H(\text{DB})$

when $\mathcal{Z}$ sends (Impersonate, sid, ssid || $prfx_C$ || req || resp_Z || h_Z , C) to $\mathcal{F}_{\text{aPAKE}}$, Sim now sends (Impersonate, sid, ssid, C) to $\mathcal{F}$ and relays the response to $\mathcal{Z}$ from $\mathcal{F}_{\text{aPAKE}}$. Otherwise, when $\mathcal{Z}$ sends (Impersonate, sid, ssid || $prfx_C$ || req || resp_Z || h_Z , C) to $\mathcal{F}_{\text{aPAKE}}$, Sim now sends (TestPwd, sid, ssid, C, $\perp$) to $\mathcal{F}_{\text{safPAKE}}$ and relays the response to $\mathcal{Z}$ from $\mathcal{F}_{\text{aPAKE}}$.

Indistinguishability: In the case $\mathcal{F}$ responds “correct guess”, there is no difference between **Hybrid 7** and **Hybrid 8** as $\mathcal{F}$ handles sessions marked fresh the same as sessions marked compromised. In the case $\mathcal{F}$ responds “wrong guess” Sim now triggers the “Else, if the record is interrupted, sample $k \leftarrow \{0, 1\}^\kappa$ ” clause in $\mathcal{F}$. In **Hybrid 7**, the outputs of honest parties with non-matching passwords were uniformly distributed through the simulation of $\mathcal{F}_{\text{aPAKE}}$, so there is no distinguishable difference in C’s output.

Finally, in the case where Sim sends TestPwd to $\mathcal{F}$, either (1) the OPRF session relied on a key different from the honest server (2) the PIR session was not consistent with the server’s database or (3) the PIR digest was not consistent with the server’s database. In all three cases, the server’s storage uniquely defines an OPRF output rw which defines a unique (pw, i) pair which, by [Lemma 4](#), defines a unique database DB and digest $H(\text{DB})$. By the determinism of the PIR scheme, resp is then uniquely defined. If any of these values are inconsistent, the client will sample a uniform input rw' to the aPAKE and both parties will output uniform keys unless C samples $rw' = rw$ which happens with negligible probability.

$$\mathbf{Dist}_{\mathcal{Z}}^{7,8} \leq \frac{1}{2^\kappa}$$

Hybrid 9: Replace Honest Client’s PIR Request

Changes to Sim: We now replace the simulated client’s PIR request $(\text{state}, \text{req}) \leftarrow \text{PIR.Request}(f_{pw}.\text{tag})$ with $(\text{state}', \text{req}') \leftarrow \text{PIR.Request}(0^\kappa)$. Note that after this hybrid, the only place where the simulator relies on the honest client’s password is as input to the OPRF stage.

Because Sim no longer uses the OPRF output $F_i(pw')$, Sim no longer samples or catalogues the OPRF output from the client's query.

Indistinguishability: Since in **Hybrid 8** Sim runs independently of the clients PIR state, indistinguishability can be seen by a direct reduction to the client privacy property of PIR (see [Section 2.1](#)).

$$\mathbf{Dist}_{\mathcal{Z}}^{8,9} = \mathbf{Adv}_{\mathcal{Z}}^{\text{PIR Privacy}}$$

Hybrid 10: Man in the Middle Attacks Against Client Sessions

Changes to Sim: Assuming that C is “aPAKE active”, then on receiving (NewKey, sid, ssid, $\|prfx_C\|\text{req}\|\text{resp}_Z\|h_Z$) or (TestAbort, sid, ssid, $\|prfx_C\|\text{req}\|\text{resp}_Z\|h_Z$) from Z to $\mathcal{F}_{\text{aPAKE}}$, if any of the following hold

- (Prefix, sid, ssid, $prfx_C$) was sent to C and (Prefix, sid, ssid, $prfx_S$) was sent to S with $prfx_C \neq prfx_S$
- Z sent (PRFReceiveComplete, sid, ssid, C, i) to $\mathcal{F}_{\text{OPRF}}$ where $i \neq S$ and $i \neq (k, 1)$ for server key k
- Z provided server PIR response resp_Z to C with $\text{resp}_Z \neq \text{resp}$
- Z provided client PIR request req_Z to S with $\text{req}_Z \neq \text{req}$
- Z provided digest h_Z to C with $h_Z \neq h$

send (TestPwd, sid, ssid, C, $\perp$) to $\mathcal{F}$. Finally, send (NewKey, sid, ssid, C, k^*) or (TestAbort, sid, ssid, C), respectively, to $\mathcal{F}$ and Sim now considers C “aPAKE completed”.

Indistinguishability: This change does not affect C's output behavior as

- The case that Z previously sent (TestPwd, sid, ssid, C, $\cdot$) to $\mathcal{F}_{\text{aPAKE}}$ has been handled, and sending an additional (TestPwd, sid, ssid, C, $\perp$) to $\mathcal{F}$ will be ignored. In this case, **Hybrid 10** and **Hybrid 9** are identical.
- Otherwise, if Z provided distinct prefixes to the OPRF session, changed the server's PIR response resp , or changed the server's digest h , the two honest parties will compute different sub-session identifiers for the aPAKE session and C will output a uniform key in both **Hybrid 10** and **Hybrid 9**.
- Similarly, if Z modified the OPRF index in the session, then as $pw, (k, 1)$ uniquely defines DB (by [Lemma 4](#)), C will compute $h' \neq h$ and will sample rw' . The two honest parties will then output independent outputs with overwhelming probability through simulation of the aPAKE.
- Finally, if $pw' \notin \Sigma_{pw}$, then as σ is reconstruction bounded (see [Section 5.1](#)), C will not reconstruct pw and will compute $h' \neq h$ as in the previous case.

$$\mathbf{Dist}_{\mathcal{Z}}^{9,10} \leq \frac{1}{2^\kappa}$$

Hybrid 11: Bookkeeping for OPRF Sessions

In this hybrid, Sim handles modifications to the OPRF session resulting in delayed-password-guess tickets being provided and spent. To accomplish this, Sim explicitly takes control of the OPRF

simulator's ticket counter tx now named delayedTests . Sim additionally keeps track of all delayed sessions in a set delayedSessions .

Changes to Sim: When Sim sends $(\text{PRFSendComplete}, \text{sid}, \text{ssid}, S)$ to $\mathcal{Z}$ from $\mathcal{F}_{\text{OPRF}}$ and $(\text{Prefix}, \text{sid}, \text{ssid}, \text{prfx}_S)$ is sent to S , if there was no corresponding client OPRF session which was sent $(\text{Prefix}, \text{sid}, \text{ssid}, \text{prfx}_C)$ with $\text{prfx}_S = \text{prfx}_C$ (i.e., the OPRF simulator would normally set $\text{tx}[S] := \text{tx}[S] + 1$), Sim now sets $\text{delayedTests} := \text{delayedTests} + 1$ and adds ssid to delayedSessions (i.e., sets $\text{delayedSessions} := \text{delayedSessions} \cup \{\text{ssid}\}$).

When $\mathcal{Z}$ sends $(\text{PRFReceiveComplete}, \text{sid}, \text{ssid}, P, i = S)$ to $\mathcal{F}_{\text{OPRF}}$ and there exists an OPRF session $(x, \text{prfx}) := \text{session}[(\text{ssid}, P)]$, if $\text{prfx} \neq \text{OK}$ and $\text{delayedTests} > 0$ (i.e., the OPRF simulator would normally set $\text{tx}[S] := \text{tx}[S] - 1$), set $\text{delayedTests} := \text{delayedTests} - 1$ and remove an arbitrary ssid' from delayedSessions .

Indistinguishability: We may note that a sub-session identifier is added to / removed from delayedSessions and delayedTests is incremented / decremented *exactly* when tx would have been incremented or decremented respectively. As such, this hybrid is only concerned with additional bookkeeping without any changes to the output or message behavior of the server. Therefore, **Hybrid 11** is perfectly indistinguishable from **Hybrid 10**.

$$\text{Dist}_{\mathcal{Z}}^{10,11} = 0$$

Hybrid 12: Handling Offline OPRF Queries

Changes to Sim: Whenever Sim receives $(\text{StorePwdFile}, \text{sid}, C, \text{pw})$ from $\mathcal{F}$, Sim stores $\text{password} := \text{pw}$ and in all cases, simulates the server relative to password . Whenever $\mathcal{Z}$ sends $(\text{OfflineEval}, \text{sid}, (k, 1), x)$ to $\mathcal{F}_{\text{OPRF}}$ for server OPRF key k or $(\text{PRFReceiveComplete}, \text{sid}, \text{ssid}, \mathcal{A}, (k, 1))$ to $\mathcal{F}_{\text{OPRF}}$ with corresponding session $(x, \text{prfx}) := \text{session}[(\text{ssid}, \mathcal{A})]$:

- If S is compromised, Sim sends $(\text{OfflineTestPwd}, \text{sid}, x)$ to $\mathcal{F}$.
- Otherwise, if no $F_{k,1}(\widetilde{\text{pw}})$ was provided to $\mathcal{Z}$ for any $\widetilde{\text{pw}} \in \Sigma_{\text{pw}}$, Sim aborts.

If $\mathcal{F}$ responds with pw , Sim sets $\text{password} := \text{pw}$. Note that this does not affect how Sim handles OfflineEval queries made on other indices.

Indistinguishability: Note that in the case that Sim overwrites password , it is always with the same value as previously inhabited the variable, so there is no change between **Hybrid 12** and **Hybrid 11** when this occurs.

If Sim aborts in **Hybrid 12**, but not in **Hybrid 11**, then $\mathcal{Z}$ queried $(\text{OfflineEval}, \text{sid}, (k, 1), x)$ or $(\text{PRFReceiveComplete}, \text{sid}, \text{ssid}, \mathcal{A}, (k, 1))$ before compromising the server and before learning $f_{\widetilde{\text{pw}}.\text{ek}}, f_{\widetilde{\text{pw}}.\text{rs}}$ for any $\widetilde{\text{pw}} \in \Sigma_{\text{pw}}$. As $f_{\widetilde{\text{pw}}.\text{tag}}, f_{\widetilde{\text{pw}}.\text{ek}}, f_{\widetilde{\text{pw}}.\text{rs}}$ are uniformly sampled, independently of the uniformly sampled k , then $\mathcal{Z}$, given the server's database, cannot distinguish k from a uniform string with non-negligible probability by the AE security of $\mathcal{E}$ (**Definition 1**). Therefore, $\mathcal{Z}$ makes such a query with negligible probability and Sim aborts with negligible probability.

The other difference between **Hybrid 12** and **Hybrid 11** is that in the case of a correct guess, $\mathcal{F}$ sets $\text{flag} := \text{compromised}$. This only affects the behavior of NewKey by changing NewKey to allow Sim to set the server's output key for interrupted server sessions. As Sim currently samples interrupted server sessions' keys through simulation of the aPAKE, in this regard, **Hybrid 12** is perfectly indistinguishable from **Hybrid 11**.

$$\text{Dist}_{\mathcal{Z}}^{11,12} \leq \frac{1}{2^\kappa} + \text{Adv}_{\mathcal{Z}}^{\text{AE}}$$

Hybrid 13: Simulate Online Server OPRF Sessions

In this hybrid, whenever $\mathcal{Z}$ gains a delayed-password-guess ticket, Sim will interrupt the corresponding sub-session in preparation for a TestPwd query. Similarly, whenever $\mathcal{Z}$ spends a delayed-password-guess ticket to learn the output of the OPRF on the server's key, Sim will extract $\mathcal{Z}$'s password guess and send a corresponding TestPwd query to $\mathcal{F}$.

Changes to Sim: Whenever Sim would set $\text{delayedTests} := \text{delayedTests} + 1$ and add ssid to delayedSessions , as it would in **Hybrid 12**, Sim sends $(\text{Interrupt}, \text{sid}, \text{ssid}, S)$ to $\mathcal{F}$. Additionally, whenever Sim would set $\text{delayedTests} := \text{delayedTests} - 1$ and remove ssid' from delayedSessions , if this is in response to a query $(\text{PRFReceiveComplete}, \text{sid}, \text{ssid}, \mathcal{A}, i = S)$, Sim sends $(\text{TestPwd}, \text{sid}, \text{ssid}, S, x)$ to $\mathcal{F}$ where $(x, \text{prfx}) := \text{session}[(\text{ssid}, \mathcal{A})]$.

If $\mathcal{F}$ responds with pw , Sim sets $\text{password} := \text{pw}$.

Indistinguishability: Note that these changes do not change Sim's behavior in the protocol and Sim still sends messages relative to the server's input password. As such, the messages in **Hybrid 13** and **Hybrid 12** are identically distributed. Regarding output behavior, $\mathcal{F}$ now sets all server sessions with non-matching OPRF prefixes interrupted. However, as the two honest parties will have disparate sub-session identifiers in the aPAKE stage, the session keys will only align in **Hybrid 12** in the case that $\text{flag} := \text{compromised}$. If $\mathcal{Z}$ makes an online query to $\mathcal{F}_{\text{OPRF}}$ (for the server's key) on $\text{pw}^* \in \Sigma_{\text{pw}}$, $\mathcal{F}$ will set $\text{flag} := \text{compromised}$ and the outputs will be distributed identically to **Hybrid 12**. If $\mathcal{Z}$ never queries $\mathcal{F}_{\text{OPRF}}$ (for the server's key) on $\text{pw}^* \in \Sigma_{\text{pw}}$, then by σ 's self-similarity (see [Section 5.1](#)), the server's input rw to the aPAKE stage will be uniform from $\mathcal{Z}$'s view and therefore the two parties will arrive at independent keys through the simulation of $\mathcal{F}_{\text{aPAKE}}$ with overwhelming probability.

Additionally, in **Hybrid 12**, $\mathcal{F}$ sets $\text{flag} := \text{compromised}$ if $\mathcal{Z}$ makes an offline query on $\text{pw}^* \in \Sigma_{\text{pw}}$ after compromising S and **Hybrid 13**'s outputs will be distributed identically to **Hybrid 12**.

$$\text{Dist}_{\mathcal{Z}}^{12,13} \leq \frac{1}{2^\kappa}$$

Hybrid 14: Simulate Server aPAKE Sessions

In this hybrid, we change Sim to no longer use the stored $\mathcal{F}_{\text{aPAKE}}$ password file in the aPAKE phase.

Changes to Sim: If S is "aPAKE active" i.e.,

- $(\text{Prefix}, \text{sid}, \text{ssid}, \text{prfx}_S)$ was sent to S from $\mathcal{F}_{\text{OPRF}}$
- $\mathcal{Z}$ provided client PIR request $\text{req}_{\mathcal{Z}}$ to S
- S sent PIR request resp to C
- Sim sent $(\text{ServerSession}, \text{sid}, \text{ssid} \parallel \text{prfx}_S \parallel \text{req}_{\mathcal{Z}} \parallel \text{resp} \parallel h, C, S)$ to $\mathcal{Z}$ from $\mathcal{F}_{\text{aPAKE}}$
- The aPAKE session is fresh

and $\mathcal{Z}$ sends $(\text{TestPwd}, \text{sid}, \text{ssid} \parallel \text{prfx}_S \parallel \text{req}_{\mathcal{Z}} \parallel \text{resp} \parallel h, S, \text{rw}^*)$ to $\mathcal{F}_{\text{aPAKE}}$:

- If an OPRF response $F_{k,1}(\text{password}) = s \parallel \text{rw}^*$ (for $s \in \{0, 1\}^{3\kappa}$) was sent to $\mathcal{A}$, set $\widetilde{\text{rw}} := \text{rw}^*$
- Otherwise, set $\widetilde{\text{rw}} := \perp$.

Finally, simulate the aPAKE session relative to server aPAKE input $\widetilde{rw}$ instead of rw .

Indistinguishability: In **Hybrid 13**, if no such OPRF response was sent to $\mathcal{Z}$, the server's aPAKE password rw is uniform from $\mathcal{Z}$'s view and $rw^* = rw$ with negligible probability. Otherwise, when $\mathcal{Z}$ made such a query, **Sim** sent $(\text{TestPwd}, \text{sid}, \text{ssid}, S, \text{password})$ to $\mathcal{F}$ and by the self similarity property of σ , $\mathcal{F}$ set $\text{flag} := \text{compromised}$. By simulation of the aPAKE stage, **Hybrid 13** and **Hybrid 14** are identically distributed.

$$\text{Dist}_{\mathcal{Z}}^{13,14} \leq \frac{1}{2^\kappa}$$

Hybrid 15: Simulate Server Storage and PIR Messages

In this hybrid, we modify **Sim** to generate the server's password file on dummy values and upon setting **password** as in the previous hybrids, equivocate the password file by programming the necessary OPRF outputs. This has the additional effect of modifying the server's PIR responses.

Changes to Sim: Whenever **Sim** would generate the server's password file, instead of generating the server's password database DB using the server's input password pw as in the real protocol, **Sim** now generates the database as follows:

1. **Sim** no longer queries $\mathcal{F}_{\text{OPRF}}$ on pw at the beginning of the protocol. Instead, **Sim** samples $rw \leftarrow \{0, 1\}^\kappa$.
2. **Sim** samples n uniform strings $f_1, \dots, f_n \leftarrow \{0, 1\}^{3\kappa}$ and stores these values $\text{params} := (\{f_1, \dots, f_n\})$.
3. For $j \in \{1, 2, 3, \dots, n\}$, **Sim** parses f_j as $f_j.\text{tag} \| f_j.\text{ek} \| f_j.\text{rs}$ each substring of length κ .
4. **Sim** then generates the database as $\text{DB} := \{f_j.\text{tag} \mapsto \mathcal{E}.\text{Enc}(f_j.\text{ek}; (k, j); f_j.\text{rs})\}$.
5. Finally, **Sim** stores $\text{file} := (\text{DB}, k)$ and the server's aPAKE storage as in the real protocol.

Additionally, whenever **Sim** sets **password** $:= pw$, **Sim** immediately (before returning any OPRF messages) runs $\text{ProgramDatabase}(\text{sid}, pw, rw, k, \{f_1, \dots, f_n\})$ for server OPRF key k .

Indistinguishability: In **Hybrid 14**, if $\mathcal{Z}$ never queries the OPRF on any $pw^* \in \Sigma_{pw}$ for index $i = (k, 1)$ or $i = S$, then all f_i will be uniform from $\mathcal{Z}$'s view in both **Hybrid 14** and **Hybrid 15**, so the databases will be identically distributed. If $\mathcal{Z}$, instead, queries the OPRF on any $pw^* \in \Sigma_{pw}$ for index $i = (k, 1)$ or $i = S$, then **password** will be set in **Hybrid 14** and as **password** is always set to the server's password pw , we may perfectly simulate the database. Therefore, **Sim** simulates **S** identically to **Hybrid 14**.

Additionally, in the case where $\mathcal{Z}$ sends **OfflineTestPwd** queries to $\mathcal{F}_{\text{aPAKE}}$, if $\mathcal{Z}$ previously queried $\mathcal{F}_{\text{OPRF}}$ on the server's password pw , $F_{k,1}(\text{password}).rw$ was programmed to rw , and we may perfectly simulate. As rw was uniformly sampled in both **Hybrid 14** and **Hybrid 15**, if $\mathcal{Z}$ sends **OfflineTestPwd** queries to $\mathcal{F}_{\text{aPAKE}}$ *before* making such a query, $\mathcal{Z}$ correctly guesses rw with the same probability in both **Hybrid 14** and **Hybrid 15**, and the two are perfectly indistinguishable.

$$\text{Dist}_{\mathcal{Z}}^{14,15} = 0$$

Hybrid 16: Man in the Middle Attacks Against Server Sessions

Changes to Sim: Assuming that **S** is “aPAKE active” and **Sim** receives $(\text{NewKey}, \text{sid}, \text{ssid} \| \text{prfx}_S \| \text{req}_{\mathcal{Z}} \| \text{resp} \| h)$ or $(\text{TestAbort}, \text{sid}, \text{ssid} \| \text{prfx}_S \| \text{req}_{\mathcal{Z}} \| \text{resp} \| h)$ from $\mathcal{Z}$ to $\mathcal{F}_{\text{aPAKE}}$, if $(\text{Prefix}, \text{sid}, \text{ssid}, \text{prfx}_C)$ was sent to **C** and $(\text{Prefix}, \text{sid}, \text{ssid}, \text{prfx}_S)$ was sent to **S** with $\text{prfx}_C = \text{prfx}_S$ then if any of the following hold:

- $\mathcal{Z}$ sent $(\text{PRFReceiveComplete}, \text{sid}, \text{ssid}, C, i)$ to $\mathcal{F}_{\text{OPRF}}$ where $i \neq s$ and $i \neq (k, 1)$ for server key k

- $\mathcal{Z}$ provided client PIR request $\text{req}_{\mathcal{Z}}$ to $\mathcal{S}$ with $\text{req}_{\mathcal{Z}} \neq \text{req}$
- $\mathcal{Z}$ provided server PIR request $\text{resp}_{\mathcal{Z}}$ to $\mathcal{C}$ with $\text{resp}_{\mathcal{Z}} \neq \text{resp}$
- $\mathcal{Z}$ provided digest $h_{\mathcal{Z}}$ to $\mathcal{C}$ with $h_{\mathcal{Z}} \neq h$

send $(\text{TestPwd}, \text{sid}, \text{ssid}, \mathcal{S}, \perp)$ to $\mathcal{F}$. Finally, send $(\text{NewKey}, \text{sid}, \text{ssid}, \mathcal{S}, k^*)$ or $(\text{TestAbort}, \text{sid}, \text{ssid}, \mathcal{S})$, respectively, to $\mathcal{F}$ and Sim now considers $\mathcal{S}$ “aPAKE completed”.

Indistinguishability: This change does not affect $\mathcal{S}$ ’s output behavior as

- The case that $\mathcal{Z}$ previously sent $(\text{TestPwd}, \text{sid}, \text{ssid}, \mathcal{S}, \cdot)$ to $\mathcal{F}_{\text{aPAKE}}$ has been handled, and sending an additional $(\text{TestPwd}, \text{sid}, \text{ssid}, \mathcal{C}, \perp)$ to $\mathcal{F}$ will be ignored. In this case, **Hybrid 16** and **Hybrid 15** are identical.
- Otherwise, if $\text{req}_{\mathcal{Z}} \neq \text{req}$ the two honest parties will arrive at disparate sub-session identifiers for the aPAKE stage and will arrive at independent and uniform keys in both **Hybrid 16** and **Hybrid 15** through simulation of the aPAKE.

$$\text{Dist}_{\mathcal{Z}}^{15,16} = 0$$

Hybrid 17: Align Fresh Session Keys

Changes to $\mathcal{F}$: We modify the NewKey interface in $\mathcal{F}$ to have an additional first and last clause:

- Let $s := \sigma(\text{sessionStatus}[(\text{sid}, \text{ssid}, \mathcal{S}, \mathcal{C})], \text{sessionStatus}[(\text{sid}, \text{ssid}, \mathcal{C}, \mathcal{S})])$
- Else, if the record is fresh, $(\text{sid}, \text{ssid}, k')$ was sent to $\mathcal{P}'$, $s \neq \perp$, and at the time $\text{sessionStatus}[(\text{sid}, \text{ssid}, \mathcal{P}', \mathcal{P})]$ was fresh, set $k := k'$.

Note that now, the only differences between $\mathcal{F}$ and $\mathcal{F}_{\text{safPAKE}}$ are

- On receiving $(\text{NewKey}, \text{sid}, \text{ssid}, \mathcal{P}, k^*)$, if the corresponding session is fresh and this is the first NewKey command for the session pair, $\mathcal{F}$ sets $k = k^*$ while $\mathcal{F}_{\text{safPAKE}}$ samples $k \leftarrow \{0, 1\}^{\kappa}$.
- On receiving $(\text{StorePwdFile}, \text{sid}, \mathcal{C}, pw)$, $\mathcal{F}$ relays the message to Sim while $\mathcal{F}_{\text{safPAKE}}$ sends no message.
- On receiving $(\text{ClientSession}, \text{sid}, \text{ssid}, \mathcal{S}, pw')$, $\mathcal{F}$ relays the full message to Sim while $\mathcal{F}_{\text{safPAKE}}$ sends the message without pw' .

Indistinguishability: As of **Hybrid 16**, all situations resulting in marking honest sessions as interrupted or compromised have been handled. As such, this change only affects honest sessions with similar passwords barring $\mathcal{Z}$ ’s interaction. By correctness of the protocol, if the two honest parties hold similar passwords (i.e., $\sigma(pw, pw') \neq \perp$), the parties will input the same rw to the aPAKE stage and, by simulation of the aPAKE for honest parties, arrive at the same session key $k = k'$.

$$\text{Dist}_{\mathcal{Z}}^{16,17} = 0$$

Hybrid 18: Random key for honest session

Changes to $\mathcal{F}$: We modify the NewKey interface in $\mathcal{F}$ to replace the following two clauses

- Else, if the record is interrupted, sample $k \leftarrow \{0, 1\}^{\kappa}$.

- Else, set $k := k^*$.

with a single clause

- If k is undefined, sample $k \leftarrow \{0, 1\}^\kappa$.

After which, the **NewKey** interface in $\mathcal{F}$ is identical to the one found in $\mathcal{F}_{\text{safPAKE}}$. This change modifies the behaviour of **fresh** sessions by uniformly sampling the key — opposed to directly setting the key — if this is the first **fresh** half of a session pair to complete.

Indistinguishability: Note that in **Hybrid 17**, if a **fresh** session is the first of a session pair to complete, the key was uniformly sampled by simulation of the aPAKE stage; therefore, **Hybrid 18**’s outputs are identically distributed in this case. In the case where a **fresh** session is the second of a session pair to complete, but the first of the pair was either **compromised**, **interrupted**, or had dissimilar passwords:

- In the case where **Sim** marks an honest session as **compromised** or **interrupted**; $\mathcal{Z}$ provided $\text{prfx}_C \neq \text{prfx}_S$, $\text{resp}_Z \neq \text{resp}$, or $\text{req}_Z \neq \text{req}$ to the corresponding parties; or **Sim** sent $(\text{PRFReceiveComplete}, \text{sid}, \text{ssid}, C, i)$ to $\mathcal{F}_{\text{OPRF}}$ where $i \neq S$ and $i \neq (k, 1)$ for server key k . In the first cases, the two honest parties will have disparate sub-session identifiers while in the second, C either sampled a uniform rw' to the aPAKE stage or used $rw' \neq rw$ as we removed such collisions in the OPRF. In all cases, the parties will arrive at independent and uniform outputs in **Hybrid 17** by simulation of the aPAKE.
- In the case where the two parties had dissimilar passwords ($\sigma(pw, pw') = \perp$), then as σ is reconstruction bounded (see [Section 5.1](#)), C did not reconstruct server input pw . Therefore, C either sampled a uniform rw' to the aPAKE stage, or used $rw' \neq rw$ as we removed such collisions in the OPRF. In both cases, the two parties’ inputs were distinct with overwhelming probability and by simulation of the aPAKE, the parties in **Hybrid 17** received uniform and independent keys.

$$\text{Dist}_{\mathcal{Z}}^{17,18} = \frac{1}{2^\kappa}$$

Hybrid 19: Generate Server Storage On Demand

In this hybrid, instead of generating the server’s storage on receiving $(\text{StorePwdFile}, \text{sid}, C, pw)$ (as $\mathcal{F}_{\text{safPAKE}}$ doesn’t relay this command) **Sim** generates them when needed.

Changes to Sim: If **Sim** receives $(\text{StorePwdFile}, \text{sid}, C, pw)$ from $\mathcal{F}_{\text{safPAKE}}$, **Sim** no longer runs server’s code to generate the password file. When **Sim** receives $(\text{StealPwdFile}, \text{sid})$ to $\mathcal{F}_{\text{aPAKE}}$ and $(\text{StealPwdFile}, \text{sid})$ to S from $\mathcal{Z}$, **Sim** sends $(\text{StealPwdFile}, \text{sid})$ to $\mathcal{F}_{\text{safPAKE}}$

- If $\mathcal{F}_{\text{safPAKE}}$ replies with “password file stolen”, if **file** is undefined, **Sim** then generates the server’s database **DB**, generates the server’s OPRF key k , and stores **file** $:= (\text{DB}, k)$. In all cases, **Sim** sends (DB, k) to $\mathcal{Z}$.
- Otherwise, **Sim** relays $\mathcal{F}_{\text{safPAKE}}$ ’s response to $\mathcal{Z}$.

When **Sim** receives $(\text{ServerSession}, \text{sid}, \text{ssid}, S)$ from $\mathcal{F}_{\text{safPAKE}}$ when **file** is undefined, **Sim** then generates the server’s database **DB**, generates the server’s OPRF key k , and stores **file** $:= (\text{DB}, k)$.

Indistinguishability: As **Sim** will still generate the required storage before it is used, **Hybrid 19** is simply a reordering of private computation for S and is perfectly indistinguishable from **Hybrid 18**.

$$\text{Dist}_{\mathcal{Z}}^{18,19} = 0$$

Hybrid 20: Remove the Client's Password

Changes to $\mathcal{F}$: We modify $\mathcal{F}$ so that upon receiving $(\text{ClientSession}, \text{sid}, \text{ssid}, S, pw')$, $\mathcal{F}$ sends $(\text{ClientSession}, \text{sid}, \text{ssid}, S)$ to Sim instead of sending the full message $(\text{ClientSession}, \text{sid}, \text{ssid}, S, pw')$. Note that this completely removes the client's password from Sim 's view.

Changes to Sim : In **Hybrid 19**, the only place where Sim continued to rely on the honest client's password was as input to the OPRF stage. We modify Sim to no longer simulate C 's OPRF query and, instead, send $(\text{PRFEval}, \text{sid}, \text{ssid}, C, S)$ to $\mathcal{Z}$ without making a corresponding query for C .

Indistinguishability: As **Hybrid 19** runs C 's PIR request, PIR verification, and OPRF verification steps independent of the OPRF output received, then this modification is perfectly indistinguishable by simulation of the OPRF.

$$\text{Dist}_{\mathcal{Z}}^{19,20} = 0$$

Hybrid 21: Remove the StorePwdFile Message

Changes to $\mathcal{F}$: We modify $\mathcal{F}$ so that upon receiving $(\text{StorePwdFile}, \text{sid}, C, pw)$, $\mathcal{F}$ sends no message to Sim — instead of relaying the message. Note that this completely removes the server's password from Sim 's view unless it receives the password from a TestPwd query.

Changes to Sim : Instead of seeding `password` with pw , Sim sets `password` $:= \perp$ at the start of simulation. Any previous check against `password` while it is $\perp$ is considered to fail. In the simulation of C against dictionary attacks where Sim compares against server password pw , Sim now compares against `password`.

Indistinguishability: In simulation of the client, the only place in **Hybrid 20** that pw is used is in verifying $\mathcal{Z}$'s input to the aPAKE session against the client. Note that in **Hybrid 20** before comparing against pw , Sim first verifies that $F_{k,1}(pw)$ has been set. If this value *has not* been set, no comparison is made and there is no difference between **Hybrid 21** and **Hybrid 20**. If $F_{k,1}(pw)$ *has* been set, then Sim has been triggered to send $(\text{TestPwd}, \text{sid}, S, pw)$ to $\mathcal{F}_{\text{saffPAKE}}$ and therefore `password` $= pw$ and there is no difference between **Hybrid 21** and **Hybrid 20**.

Note that the last place Sim uses the server's password **Hybrid 20** is by seeding `password`. Any test against `password` in **Hybrid 20** is done through checking if $F_{k,1}(\text{password})$ is set. If $F_{k,1}(pw)$ was previously sent to $\mathcal{Z}$, Sim was triggered to send TestPwd or a post-compromise OfflineTestPwd on pw and set `password` $:= pw$. As this query happens before any OPRF output is sampled, Sim acted independently of the original state of `password` in **Hybrid 20**.

$$\text{Dist}_{\mathcal{Z}}^{20,21} = 0$$

Hybrid 22: Ideal World

In **Hybrid 21**, we have that $\mathcal{F} = \mathcal{F}_{\text{saffPAKE}}$. Further, the total distinguishing advantage $\mathcal{Z}$ has across all hybrids is

$$\begin{aligned} \text{Adv}_{\mathcal{Z}}^{\text{Real, Ideal}} &\leq \sum_i \text{Adv}_{\mathcal{Z}}^{i, i+1} \\ &\leq \frac{4 \cdot q_{\text{OPRF}}^2 + q_{\text{RO}}^2 + 8}{2^\kappa} + \text{Adv}_{\mathcal{Z}}^{\text{PIR Privacy}} + 2\text{Adv}_{\mathcal{Z}}^{\text{AE}} \end{aligned}$$

which is negligible in our security parameter κ and our theorem follows. $\square$

B aPAKE Functionality

We recall the standard UC asymmetric PAKE functionality [GMR06, Hes20].

Functionality $\mathcal{F}_{\text{aPAKE}}$
Parameters: Security parameter κ, client C, server S, and ideal adversary $\mathcal{A}^*$ Storage: Five maps: <code>fileStatus</code>, <code>file</code>, <code>sessionStatus</code>, <code>offline</code>, and <code>session</code> Password Registration Upon receiving <code>(StorePwdFile, sid, C, pw)</code> from S, if <code>file[(sid, C, S)]</code> is undefined, set <code>file[(sid, C, S)] := pw</code> and set <code>fileStatus[(sid, C, S)] := uncompromised</code>. Stealing Password Data Upon receiving <code>(StealPwdFile, sid)</code> from $\mathcal{A}^*$: 1. If <code>file[(sid, C, S)]</code> is undefined, return “no password file” to $\mathcal{A}^*$. 2. Otherwise: 1. If <code>file[(sid, C, S)] ∈ offline[sid]</code>, send <code>file[(sid, C, S)]</code> to $\mathcal{A}^*$. 2. Otherwise, return “password file stolen” to $\mathcal{A}^*$. 3. In both cases, if <code>fileStatus[(sid, C, S)] = uncompromised</code>, set <code>fileStatus[(sid, C, S)] := stolen</code>. Upon receiving <code>(OfflineTestPwd, sid, pw*)</code> from $\mathcal{A}^*$: 1. If <code>fileStatus[(sid, C, S)] = stolen</code>: 1. Retrieve <code>pw := file[(sid, C, S)]</code>. 2. If <code>pw = pw*</code>: return “correct guess” to $\mathcal{A}^*$. 3. Otherwise, return “wrong guess” to $\mathcal{A}^*$. 2. Otherwise, add <code>pw*</code> to <code>offline[sid]</code>. Password Authentication Upon receiving <code>(ClientSession, sid, ssid, S, pw')</code> from C: 1. Send <code>(ClientSession, sid, ssid, C, S)</code> to $\mathcal{A}^*$. 2. If <code>sessionStatus[(sid, ssid, C, S)]</code> is undefined, set <code>session[(sid, ssid, C, S)] := pw'</code> and set <code>sessionStatus[(sid, ssid, C, S)] := fresh</code>. Upon receiving <code>(ServerSession, sid, ssid)</code> from S: 1. If <code>file[(sid, C, S)]</code> is undefined, ignore this query. 2. Otherwise, retrieve <code>pw := file[(sid, C, S)]</code> and send <code>(ServerSession, sid, ssid, C, S)</code> to $\mathcal{A}^*$. 3. If <code>sessionStatus[(sid, ssid, S, C)]</code> is undefined, set <code>session[(sid, ssid, S, C)] := pw</code> and set <code>sessionStatus[(sid, ssid, S, C)] := fresh</code>.

Figure 6: Ideal functionality $\mathcal{F}_{\text{aPAKE}}$ (part 1)

Active Session Attacks

Upon receiving (TestPwd, sid, ssid, P, pw^*) from $\mathcal{A}^*$:

1. If $\text{sessionStatus}[(\text{sid}, \text{ssid}, P, P')]$ is undefined, ignore this query.
2. Otherwise, retrieve $pw' := \text{session}[(\text{sid}, \text{ssid}, P, P')]$.
3. If $\text{sessionStatus}[(\text{sid}, \text{ssid}, P, P')] = \text{fresh}$:
 1. If $pw' = pw^*$, return “correct guess” to $\mathcal{A}^*$ and set $\text{sessionStatus}[(\text{sid}, \text{ssid}, P, P')] := \text{compromised}$.
 2. Otherwise, set $\text{sessionStatus}[(\text{sid}, \text{ssid}, P, P')] := \text{interrupted}$ and return “wrong guess” to $\mathcal{A}^*$.

Upon receiving (Impersonate, sid, ssid) from $\mathcal{A}^*$:

1. If $\text{sessionStatus}[(\text{sid}, \text{ssid}, C, S)] = \text{fresh}$:
 1. If $\text{fileStatus}[(\text{sid}, C, S)] = \text{stolen}$ and $\text{file}[(\text{sid}, C, S)] = \text{session}[(\text{sid}, \text{ssid}, C, S)]$, set $\text{sessionStatus}[(\text{sid}, \text{ssid}, C, S)] := \text{compromised}$ and return “correct guess” to $\mathcal{A}^*$.
 2. Otherwise set $\text{sessionStatus}[(\text{sid}, \text{ssid}, C, S)] := \text{interrupted}$ and return “wrong guess” to $\mathcal{A}^*$.

Key Generation and Authentication

Upon receiving (NewKey, sid, ssid, P, k^*) from $\mathcal{A}^*$ where $|k^*| = \kappa$:

1. If $\text{sessionStatus}[(\text{sid}, \text{ssid}, P, P')]$ is defined, but is not completed:
 1. If the record is compromised, set $k := k^*$.
 2. Else, if the record is fresh, $(\text{sid}, \text{ssid}, k')$ was sent to P' , $\text{session}[(\text{sid}, \text{ssid}, P, P')] = \text{session}[(\text{ssid}, P', P)]$, and at the time $\text{sessionStatus}[(\text{sid}, \text{ssid}, P', P)]$ was fresh, set $k := k'$.
 3. Otherwise, sample $k \leftarrow \{0, 1\}^\kappa$.
2. Finally, set $\text{sessionStatus}[(\text{sid}, \text{ssid}, P, P')] := \text{completed}$ and send $(\text{sid}, \text{ssid}, k)$ to P.

Upon receiving (TestAbort, sid, ssid, P) from $\mathcal{A}^*$:

1. If $\text{sessionStatus}[(\text{sid}, \text{ssid}, P, P')]$ is defined, but is not completed:
 1. Retrieve $pw' := \text{session}[(\text{sid}, \text{ssid}, P, P')]$.
 2. If the record is fresh and $\text{session}[(\text{sid}, \text{ssid}, P', P)] = pw'$ send “success” to $\mathcal{A}^*$.
 3. Else, send “abort” to $\mathcal{A}^*$ and (Abort, sid, ssid) to P.
 4. In both cases, set $\text{sessionStatus}[(\text{sid}, \text{ssid}, P, P')] := \text{completed}$.

Figure 7: Ideal functionality $\mathcal{F}_{\text{aPAKE}}$ (part 2)